



Threagile

Agile Threat Modeling

Threat Model Report

VaultNote Threat Model

18 April 2026

VaultNote Security Team

Table of Contents

Results Overview

Management Summary	4
Impact Analysis of 28 Initial Risks in 6 Categories	5
Risk Mitigation	6
Asset Register	7
Impact Analysis of 28 Remaining Risks in 6 Categories	12
Application Overview	13
Data-Flow Diagram	14
Security Requirements	16
Abuse Cases	17
Tag Listing	18
STRIDE Classification of Identified Risks	23
Assignment by Function	25
RAA Analysis	27
Data Mapping	30
Out-of-Scope Assets: 0 Assets	31
Potential Model Failures: 0 / 0 Risks	32
Questions: 0 / 0 Questions	33

Risks by Vulnerability category

Identified Risks by Vulnerability category	34
Data Disclosure - Personal Data Transmitted Over Unencrypted Channel: 2 / 2 Risks	35
Detectability - PII Flows Without Audit Logging: 6 / 6 Risks	37
Identifiability - Personal Data Stored Without Encryption at Rest: 6 / 6 Risks	39
Non-Repudiation - PII or Auth Tokens Accessible in Browser-Side Storage: 1 / 1 Risk	41
Unawareness - PII Processed Without a Consent Management Component: 1 / 1 Risk	44
Linkability - Same PII Data Asset Processed Across Multiple Trust Boundaries: 12 / 12 Risks	46

Risks by Technical Asset

Identified Risks by Technical Asset	49
API Server: 3 / 3 Risks	50
AWS RDS PostgreSQL: 2 / 2 Risks	54
AWS S3 Notes Bucket: 2 / 2 Risks	56
Browser SPA: 3 / 3 Risks	58
ECS Container Platform: 2 / 2 Risks	61
LLM Note Summarizer: 1 / 1 Risk	63
Lambda Email Notifier: 1 / 1 Risk	66
MinIO Object Storage: 2 / 2 Risks	68

Nginx Reverse Proxy: 3 / 3 Risks	70
Note RAG Pipeline: 2 / 2 Risks	73
PostgreSQL Database: 2 / 2 Risks	76
Redis Cache: 2 / 2 Risks	78
VaultNote Vector Store: 2 / 2 Risks	80
AWS API Gateway: 1 / 1 Risk	83
AWS ECR Container Registry: 0 / 0 Risks	85
AWS SES: 0 / 0 Risks	88
GitHub Actions Build Pipeline: 0 / 0 Risks	90
VaultNote Source Repository: 0 / 0 Risks	93

Data Breach Probabilities by Data Asset

Identified Data Breach Probabilities by Data Asset	95
AI Model Responses: 1 / 1 Risk	96
Build Artifacts: 2 / 2 Risks	97
Embedding Vectors: 5 / 5 Risks	98
File Attachments: 14 / 14 Risks	99
Note Content: 21 / 21 Risks	100
Notification Payloads: 1 / 1 Risk	102
Session Tokens: 5 / 5 Risks	103
User Credentials: 16 / 16 Risks	104

Trust Boundaries

AI Services	105
Application Network	105
AWS Cloud	105
Data Tier	106
DMZ	106
External CI/CD	106
Internet	106

Shared Runtime

Docker Host	108
-------------	-----

About Threagile

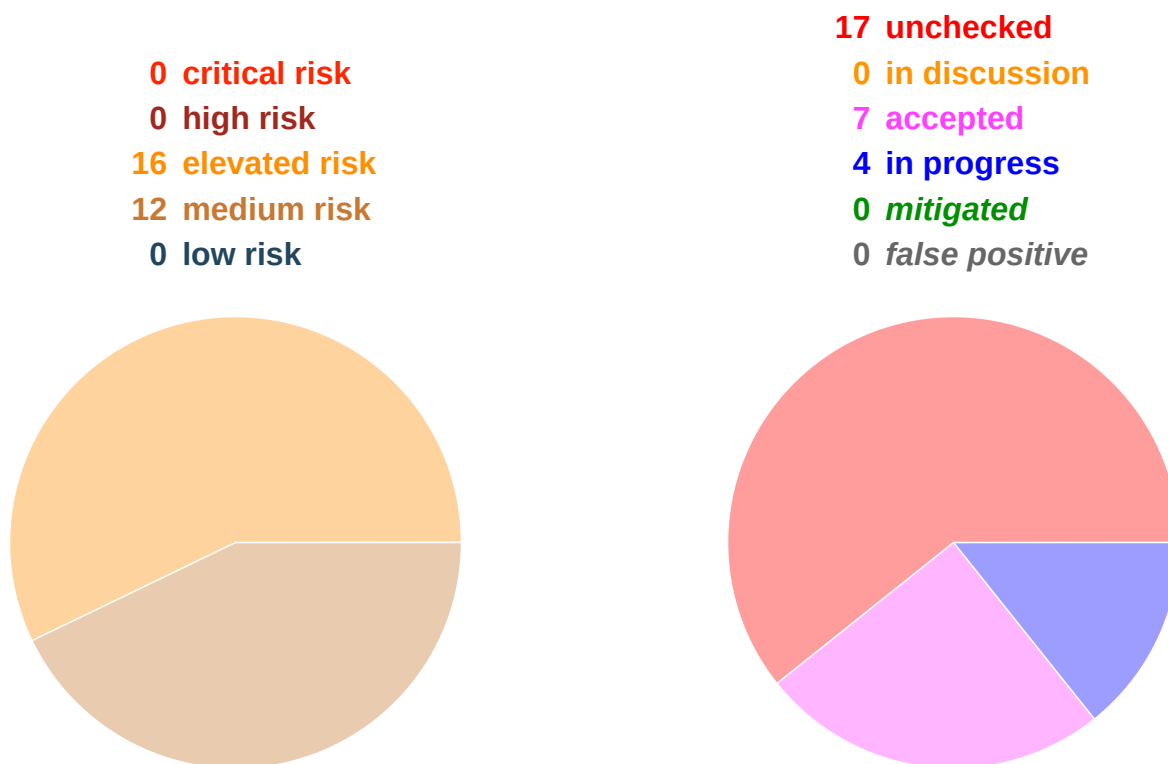
Risk Rules Checked by Threagile	109
Disclaimer	110

Management Summary

Threagile toolkit was used to model the architecture of "VaultNote Threat Model" and derive risks by analyzing the components and data flows. The risks identified during this analysis are shown in the following chapters. Identified risks during threat modeling do not necessarily mean that the vulnerability associated with this risk actually exists: it is more to be seen as a list of potential risks and threats, which should be individually reviewed and reduced by removing false positives. For the remaining risks it should be checked in the design and implementation of "VaultNote Threat Model" whether the mitigation advices have been applied or not.

Each risk finding references a chapter of the OWASP ASVS (Application Security Verification Standard) audit checklist. The OWASP ASVS checklist should be considered as an inspiration by architects and developers to further harden the application in a Defense-in-Depth approach. Additionally, for each risk finding a link towards a matching OWASP Cheat Sheet or similar with technical details about how to implement a mitigation is given.

In total **28 initial risks** in **6 categories** have been identified during the threat modeling process:



VaultNote is a containerized note-taking application comprised of a React SPA, Nginx reverse proxy, Node.js/Express REST API, PostgreSQL database, Redis session store, and MinIO object storage. The architecture deliberately contains several intentional misconfigurations (unencrypted internal HTTP, no encryption at rest, hardcoded credentials) to demonstrate automated threat detection via Threagile. All findings below are expected and serve as demonstration targets.

Impact Analysis of 28 Initial Risks in 6 Categories

The most prevalent impacts of the **28 initial risks** (distributed over **6 risk categories**) are (taking the severity ratings into account and using the highest for each category):

Risk finding paragraphs are clickable and link to the corresponding chapter.

Elevated: **Data Disclosure - Personal Data Transmitted Over Unencrypted Channel**: 2 Initial Risks - Exploitation likelihood is *Likely with High* impact.

Exposure of personal data to eavesdroppers, potential data breach notification obligations under GDPR Article 33, and significant regulatory penalties.

Elevated: **Detectability - PII Flows Without Audit Logging**: 6 Initial Risks - Exploitation likelihood is *Likely with Medium* impact.

Inability to detect unauthorized access to personal data, failure to meet GDPR accountability obligations, and potential inability to meet breach notification deadlines.

Elevated: **Identifiability - Personal Data Stored Without Encryption at Rest**: 6 Initial Risks - Exploitation likelihood is *Likely with High* impact.

Physical or logical access to unencrypted volumes directly exposes personal data, enabling identification of natural persons and constituting a reportable data breach.

Elevated: **Non-Repudiation - PII or Auth Tokens Accessible in Browser-Side Storage**: 1 Initial Risk - Exploitation likelihood is *Likely with Medium* impact.

Stolen session tokens allow account takeover and impersonation. The original user cannot repudiate actions performed by the attacker. Stolen PII violates GDPR data minimisation and storage limitation principles.

Elevated: **Unawareness - PII Processed Without a Consent Management Component**: 1 Initial Risk - Exploitation likelihood is *Likely with Medium* impact.

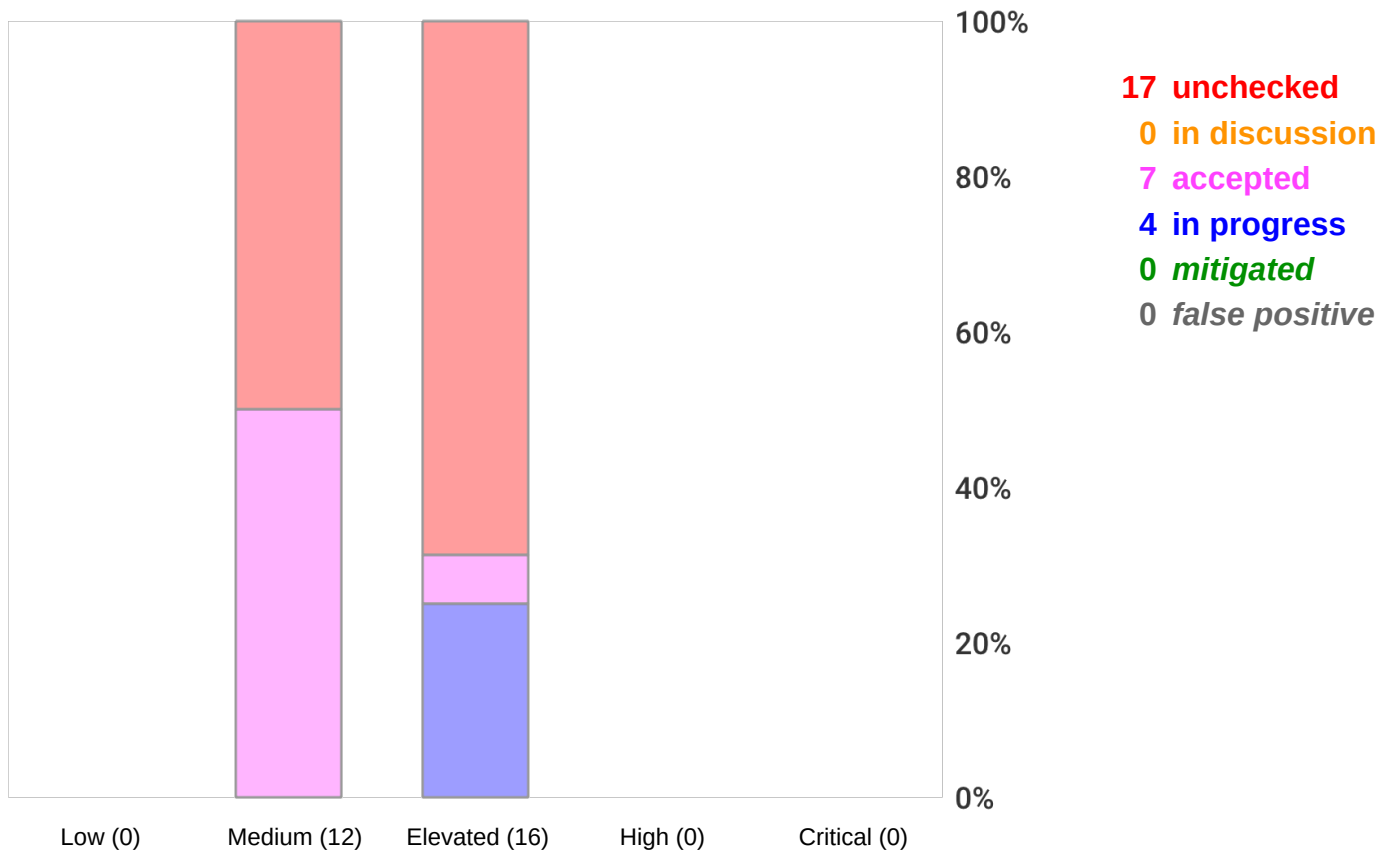
Data subjects cannot exercise GDPR rights. Regulatory investigation risk and class-action exposure in jurisdictions with private right of action.

Medium: **Linkability - Same PII Data Asset Processed Across Multiple Trust Boundaries**: 12 Initial Risks - Exploitation likelihood is *Unlikely with Medium* impact.

Aggregation of personal data enables profiling, re-identification, and violations of GDPR data minimisation and purpose limitation principles (Articles 5(1)(b) and 5(1)(c)).

Risk Mitigation

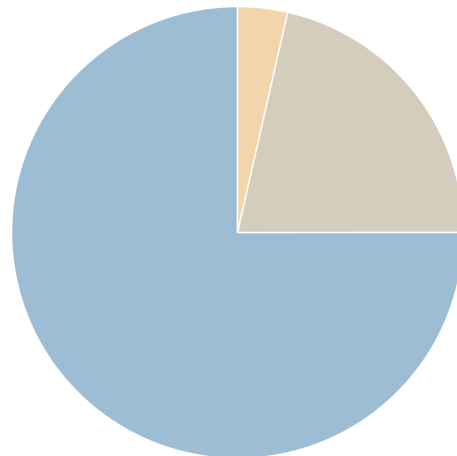
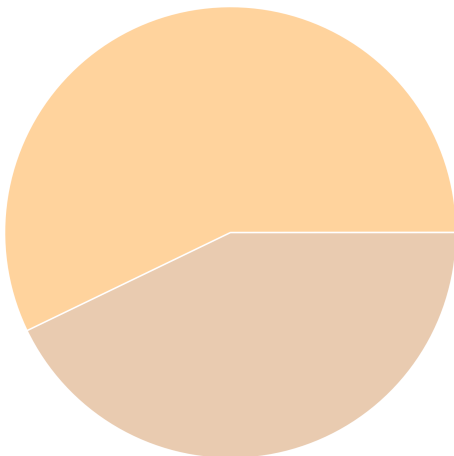
The following chart gives a high-level overview of the risk tracking status (including mitigated risks):



After removal of risks with status *mitigated* and *false positive* the following 28 remain unmitigated:

0 unmitigated critical risk
0 unmitigated high risk
16 unmitigated elevated risk
12 unmitigated medium risk
0 unmitigated low risk

0 business side related
21 architecture related
1 development related
6 operations related



Asset Register

Technical Assets

API Server

Node.js/Express REST API. Handles authentication (JWT/bcrypt), CRUD on notes, file uploads to MinIO. Bridges the frontend and backend networks. Single most security-critical component — compromise equals full data access.

AWS API Gateway

AWS API Gateway (HTTP API) fronting the ECS API containers in production. Routes traffic from the ALB to backend ECS tasks. INTENTIONAL: No usage plan or throttling policy configured. No WAF association on the API gateway stage. Auth relies entirely on the API server (JWT) — no gateway-level auth enforcement.

AWS ECR Container Registry

AWS Elastic Container Registry storing VaultNote Docker images (api, nginx). INTENTIONAL: Image vulnerability scanning is disabled (ECR Basic scanning only, not Enhanced/Inspector). Images are not signed — no Cosign/Sigstore policy enforced. Repository is private but no admission controller validates signatures at deploy time.

AWS RDS PostgreSQL

AWS RDS PostgreSQL instance (production). Replaces the Docker-based PostgreSQL in production deployments. Automated backups enabled (7-day retention). INTENTIONAL: Encryption at rest is disabled (storage_encrypted = false in Terraform). Public accessibility is off but subnet group is in app subnet (not data subnet), creating potential exposure via compromised app instances.

AWS S3 Notes Bucket

AWS S3 bucket storing file attachments for production deployments. Replaces MinIO S3-compatible storage in the cloud environment. Bucket is private (Block Public Access enabled) but server-side encryption is not configured. INTENTIONAL: SSE-S3 or SSE-KMS not enabled. Objects are stored unencrypted at the storage layer — provider-side access exposes raw data.

AWS SES

AWS Simple Email Service used for transactional email delivery (note-share invitations, account verification). Managed service — not custom-developed. SES sending identity (domain) is verified; DKIM and SPF records are configured. No dedicated sending quota alarm; a compromised Lambda could exhaust the daily send limit and cause notification delivery failure.

Browser SPA

React Single Page Application served from Nginx. Users authenticate, manage notes, and upload files through this interface.

ECS Container Platform

AWS ECS (Fargate) running the API and Nginx containers in production. INTENTIONAL: ECR Enhanced Scanning (Inspector v2) is not enabled. Running containers are not scanned post-deployment for newly disclosed CVEs. Container images are pulled from ECR without

signature verification.

GitHub Actions Build Pipeline

GitHub Actions CI/CD pipeline. Builds Docker images, runs tests, pushes to ECR. INTENTIONAL: No SBOM generation (no Syft or CycloneDX step configured). No build provenance attestation (no SLSA provenance step). Hardcoded `AWS_SECRET_ACCESS_KEY` in workflow environment variables.

LLM Note Summarizer

OpenAI-compatible LLM inference endpoint used to generate AI summaries of user notes on demand. Deployed as a containerised model proxy (LiteLLM) with a public API endpoint for testing. INTENTIONAL: Endpoint is internet-accessible and lacks authentication — the has-authentication tag is intentionally absent to demonstrate the finding. No adversarial input shield (Llama Guard or guardrails) is configured. System prompt contains internal business context visible to adversarial probing.

Lambda Email Notifier

AWS Lambda function that sends transactional emails (note-share invitations, account verification) triggered by SQS messages from the API server. Processes user email addresses (PII) and note titles in notification payloads. INTENTIONAL: No dead letter queue (DLQ) configured on the SQS event source. Failed notifications are silently dropped after 3 retries with no audit trail.

MinIO Object Storage

S3-compatible object store for file attachments. INTENTIONAL MISCONFIGURATION: Uses default minioadmin/minioadmin credentials (visible in docker-compose.yml) and stores files without encryption at rest. HTTP-only (no TLS) within the Docker network.

Nginx Reverse Proxy

Nginx terminates TLS and forwards all API traffic to the Express server over plain HTTP (port 3000). This is the DMZ entry point.

Note RAG Pipeline

Retrieval-Augmented Generation pipeline that assembles context chunks from the vector store before forwarding to the LLM. Built on LangChain. INTENTIONAL: No input validation or injection guard on retrieved chunks (has-input-validation absent). A note containing prompt injection payloads could be retrieved as context and alter LLM behavior. No retrieval filtering — all results from the vector store are injected into context without relevance threshold or content sanitisation.

PostgreSQL Database

Primary relational datastore. Contains users table (email + bcrypt password hash), notes table (title + content), and attachments metadata. INTENTIONAL MISCONFIGURATION: No encryption at rest (encryption: none).

Redis Cache

In-memory store used as session store and JWT revocation list. INTENTIONAL MISCONFIGURATION: No encryption at rest. If Redis persistence is enabled and the RDB file is exfiltrated, session tokens are exposed.

VaultNote Source Repository

GitHub repository hosting the VaultNote monorepo (API, frontend, Terraform). Contains application secrets in compose files (minioadmin credentials visible in git history) and Terraform state references. INTENTIONAL: No branch protection rules configured — developers push directly to main, bypassing review and signed-commit requirements.

VaultNote Vector Store

Qdrant vector database storing embedding vectors for semantic search and RAG context retrieval. Vectors are generated by the embedding service from user note content. INTENTIONAL: No encryption at rest configured (encryption: none). The vector store is queryable by the LLM pipeline and the API server without row-level isolation — all user embeddings share the same collection. No access control list separates per-user embedding queries.

Data Assets

AI Model Responses

LLM-generated summaries and explanations of user notes. May contain paraphrased versions of sensitive note content. Cached temporarily in the API response layer.

Build Artifacts

Docker container images, compiled source bundles, and dependency lockfiles produced by the CI/CD pipeline. Integrity is critical — these are the deployable units of the application.

Embedding Vectors

High-dimensional vector representations of user note content generated by the embedding service. Semantically encodes note information even without storing the original text.

File Attachments

Binary file uploads attached to notes. May include documents, images, or other sensitive files depending on note usage.

Note Content

User-authored note text — may include personal, financial, or health information depending on how users employ the application.

Notification Payloads

Email notification payloads containing user email addresses and note titles sent via Lambda to AWS SES. Contains PII (email address) and indirect note content references.

Session Tokens

JWT tokens and Redis session keys used to maintain authenticated state. Theft allows session hijacking without knowing the user's password.

User Credentials

Email addresses and bcrypt-hashed passwords used for authentication. Compromise allows full account takeover.

Impact Analysis of 28 Remaining Risks in 6 Categories

The most prevalent impacts of the **28 remaining risks** (distributed over **6 risk categories**) are (taking the severity ratings into account and using the highest for each category):

Risk finding paragraphs are clickable and link to the corresponding chapter.

Elevated: **Data Disclosure - Personal Data Transmitted Over Unencrypted Channel: 2 Remaining Risks** - Exploitation likelihood is *Likely with High* impact.

Exposure of personal data to eavesdroppers, potential data breach notification obligations under GDPR Article 33, and significant regulatory penalties.

Elevated: **Detectability - PII Flows Without Audit Logging: 6 Remaining Risks** - Exploitation likelihood is *Likely with Medium* impact.

Inability to detect unauthorized access to personal data, failure to meet GDPR accountability obligations, and potential inability to meet breach notification deadlines.

Elevated: **Identifiability - Personal Data Stored Without Encryption at Rest: 6 Remaining Risks** - Exploitation likelihood is *Likely with High* impact.

Physical or logical access to unencrypted volumes directly exposes personal data, enabling identification of natural persons and constituting a reportable data breach.

Elevated: **Non-Repudiation - PII or Auth Tokens Accessible in Browser-Side Storage: 1 Remaining Risk** - Exploitation likelihood is *Likely with Medium* impact.

Stolen session tokens allow account takeover and impersonation. The original user cannot repudiate actions performed by the attacker. Stolen PII violates GDPR data minimisation and storage limitation principles.

Elevated: **Unawareness - PII Processed Without a Consent Management Component: 1 Remaining Risk** - Exploitation likelihood is *Likely with Medium* impact.

Data subjects cannot exercise GDPR rights. Regulatory investigation risk and class-action exposure in jurisdictions with private right of action.

Medium: **Linkability - Same PII Data Asset Processed Across Multiple Trust Boundaries: 12 Remaining Risks** - Exploitation likelihood is *Unlikely with Medium* impact.

Aggregation of personal data enables profiling, re-identification, and violations of GDPR data minimisation and purpose limitation principles (Articles 5(1)(b) and 5(1)(c)).

Application Overview

Business Criticality

The overall business criticality of "VaultNote Threat Model" was rated as:

(archive | operational | **IMPORTANT** | critical | mission-critical)

Business Overview

VaultNote allows authenticated users to create, edit, and share encrypted notes with file attachments. The application stores sensitive user data and note content that must be protected against unauthorized disclosure.

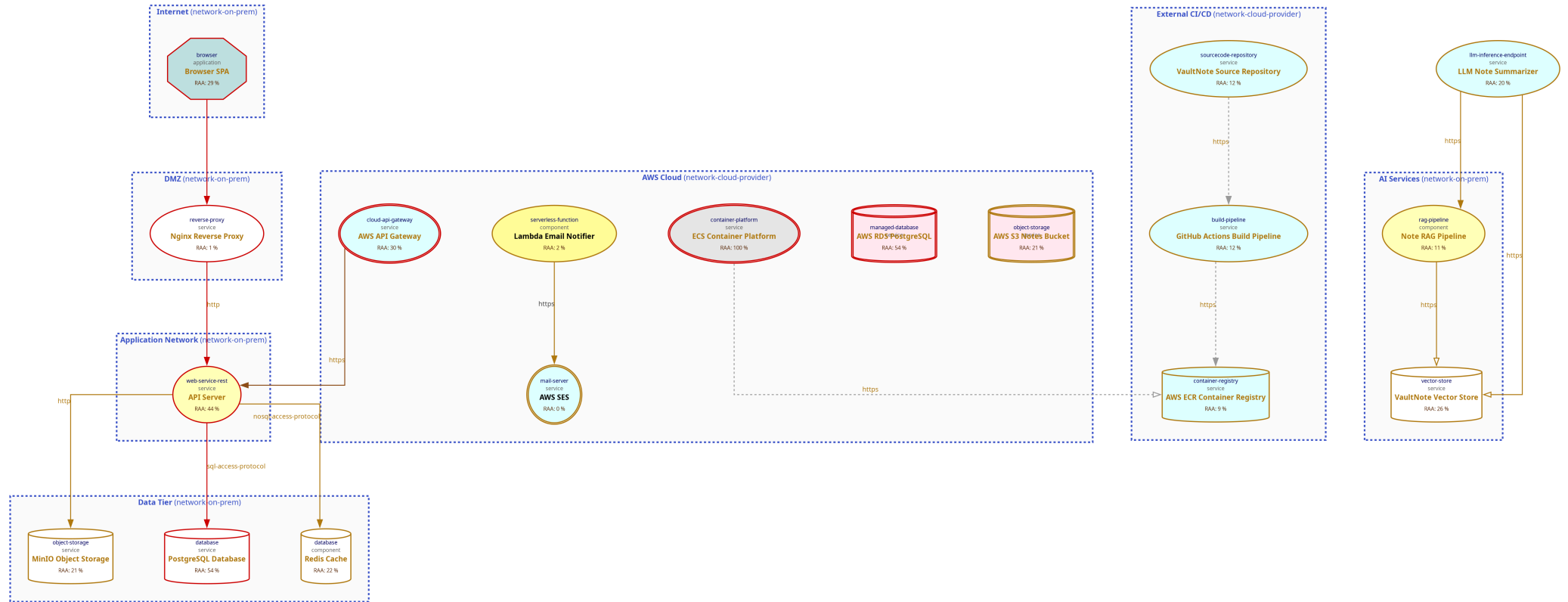
Technical Overview

Multi-tier containerized deployment: Nginx terminates TLS and proxies requests to the API over unencrypted HTTP. The API accesses PostgreSQL, Redis, and MinIO over plaintext TCP/HTTP within an internal Docker network.

Data-Flow Diagram

The following diagram was generated by Threagile based on the model input and gives a high-level overview of the data-flow between technical assets. The RAA value is the calculated *Relative Attacker Attractiveness* in percent. For a full high-resolution version of this diagram please refer to the PNG image file alongside this report.

Data-Flow Diagram - VaultNote Threat Model



Security Requirements

This chapter lists the custom security requirements which have been defined for the modeled target.

This list is not complete and regulatory or law relevant security requirements have to be taken into account as well. Also custom individual security requirements might exist for the project.

Abuse Cases

This chapter lists the custom abuse cases which have been defined for the modeled target.

This list is not complete and regulatory or law relevant abuse cases have to be taken into account as well. Also custom individual abuse cases might exist for the project.

Tag Listing

This chapter lists what tags are used by which elements.

ai

LLM Note Summarizer, Note RAG Pipeline, VaultNote Vector Store, AI Model Responses, Embedding Vectors, AI Services

api-gateway

AWS API Gateway

aws

AWS API Gateway, AWS ECR Container Registry, AWS RDS PostgreSQL, AWS S3 Notes Bucket, AWS SES, ECS Container Platform, Lambda Email Notifier, AWS Cloud

cache

Redis Cache

ci

GitHub Actions Build Pipeline

cloud-native

AWS API Gateway, AWS RDS PostgreSQL, AWS S3 Notes Bucket, AWS SES, ECS Container Platform, LLM Note Summarizer, Lambda Email Notifier, AWS Cloud

container-runtime

Docker Host

criticality-high

AWS RDS PostgreSQL, AWS S3 Notes Bucket, MinIO Object Storage, PostgreSQL Database, Redis Cache, VaultNote Vector Store

docker

Docker Host

ecr

AWS ECR Container Registry

ecs

ECS Container Platform

email

AWS SES

embeddings

VaultNote Vector Store, Embedding Vectors

express

API Server

fargate

ECS Container Platform

git

VaultNote Source Repository

github

VaultNote Source Repository

github-actions

GitHub Actions Build Pipeline

has-automated-backup

AWS RDS PostgreSQL

information-asset-container

AWS RDS PostgreSQL, AWS S3 Notes Bucket, MinIO Object Storage, PostgreSQL Database, Redis Cache, VaultNote Vector Store

intentional-misconfiguration

MinIO File Storage, MinIO Object Storage, HTTP to API Server

jwt

API Server

lambda

Lambda Email Notifier

langchain

Note RAG Pipeline

llm

LLM Note Summarizer

minio

MinIO Object Storage

nginx

Nginx Reverse Proxy

nodejs

API Server

notifications

Lambda Email Notifier

object-storage

MinIO Object Storage

openai

LLM Note Summarizer

pii

Notification Payloads

postgresql

AWS RDS PostgreSQL, PostgreSQL Database

qdrant

VaultNote Vector Store

rag

Note RAG Pipeline

rds

AWS RDS PostgreSQL

react

Browser SPA

redis

Redis Cache

relational

PostgreSQL Database

s3

AWS S3 Notes Bucket, MinIO Object Storage

ses

AWS SES

session-store

Redis Cache

spa

Browser SPA

supply-chain

AWS ECR Container Registry, GitHub Actions Build Pipeline, VaultNote Source Repository, Build Artifacts, External CI/CD

third-party-shared

AWS S3 Notes Bucket, MinIO Object Storage, PostgreSQL Database

tls-termination

Nginx Reverse Proxy

trike-actor

Browser SPA

trike-actor-untrusted

Browser SPA

trike-trust-level-admin

API Server

STRIDE Classification of Identified Risks

This chapter clusters and classifies the risks by STRIDE categories: In total **28 potential risks** have been identified during the threat modeling process of which **0 in the Spoofing** category, **0 in the Tampering** category, **6 in the Repudiation** category, **22 in the Information Disclosure** category, **0 in the Denial of Service** category, and **0 in the Elevation of Privilege** category.

Risk finding paragraphs are clickable and link to the corresponding chapter.

Spoofing

n/a

Tampering

n/a

Repudiation

Elevated: **Detectability - PII Flows Without Audit Logging: 6 / 6 Risks** - Exploitation likelihood is *Likely with Medium impact*.

Communication links that carry personal data without audit logging enabled mean that access to and use of personal data cannot be detected or reviewed after the fact. GDPR Article 5(2) requires demonstrable accountability.

Information Disclosure

Elevated: **Data Disclosure - Personal Data Transmitted Over Unencrypted Channel: 2 / 2 Risks** - Exploitation likelihood is *Likely with High impact*.

Personal data in transit over unencrypted communication links (plain HTTP, unencrypted database protocols, etc.) is exposed to network observers. GDPR Article 32 requires appropriate technical measures including encryption in transit.

Elevated: **Identifiability - Personal Data Stored Without Encryption at Rest: 6 / 6 Risks** - Exploitation likelihood is *Likely with High impact*.

Datastores holding personal data without encryption at rest allow anyone with access to the underlying storage to read and identify data subjects. GDPR Article 32 requires appropriate security measures including encryption at rest.

Elevated: **Non-Repudiation - PII or Auth Tokens Accessible in Browser-Side Storage: 1 / 1 Risk** - Exploitation likelihood is *Likely with Medium impact*.

Browser-based clients (SPAs, web apps) that process personal data or authentication tokens typically persist them in localStorage or sessionStorage. This browser-accessible storage is readable by any JavaScript running in the same origin – including injected scripts. Without HttpOnly cookies or a strict Content-Security-Policy, the stored tokens and PII are one XSS

payload away from full exfiltration. From a LINDDUN perspective this enables Non-Repudiation attacks: an adversary who steals a token can act as the victim, making it impossible to distinguish attacker actions from legitimate user actions in audit logs.

Elevated: Unawareness - PII Processed Without a Consent Management Component: 1 / 1 Risk - Exploitation likelihood is *Likely* with *Medium* impact.

When a system processes personal data from end users but has no consent-management technology in the architecture, data subjects are likely unaware of the nature and extent of processing. GDPR Articles 13 and 14 require transparent disclosure.

Medium: Linkability - Same PII Data Asset Processed Across Multiple Trust Boundaries: 12 / 12 Risks - Exploitation likelihood is *Unlikely* with *Medium* impact.

When the same personal-data asset is processed by technical assets in two or more different trust boundaries, an attacker who compromises any one boundary gains linkability across all others, enabling cross-context profiling.

Denial of Service

n/a

Elevation of Privilege

n/a

Assignment by Function

This chapter clusters and assigns the risks by functions which are most likely able to check and mitigate them: In total **28 potential risks** have been identified during the threat modeling process of which **0 should be checked by Business Side**, **21 should be checked by Architecture**, **1 should be checked by Development**, and **6 should be checked by Operations**.

Risk finding paragraphs are clickable and link to the corresponding chapter.

Business Side

n/a

Architecture

Elevated: **Data Disclosure - Personal Data Transmitted Over Unencrypted Channel: 2 / 2 Risks** - Exploitation likelihood is *Likely with High impact*.

Migrate all communication links carrying PII data assets to TLS 1.2+ (preferably TLS 1.3). For internal service-to-service communication, consider mutual TLS (mTLS).

Elevated: **Identifiability - Personal Data Stored Without Encryption at Rest: 6 / 6 Risks** - Exploitation likelihood is *Likely with High impact*.

Enable disk-level or database-level encryption on every datastore asset that stores PII data assets. Use full-disk encryption (LUKS, AWS EBS encryption, Azure SSE) or database-level TDE (pgcrypto, SQL Server TDE).

Elevated: **Unawareness - PII Processed Without a Consent Management Component: 1 / 1 Risk** - Exploitation likelihood is *Likely with Medium impact*.

Introduce a consent-manager (tagged with technology consent-manager) to capture, store, and honour data-subject preferences.

Medium: **Linkability - Same PII Data Asset Processed Across Multiple Trust Boundaries: 12 / 12 Risks** - Exploitation likelihood is *Unlikely with Medium impact*.

Use per-context pseudonyms or pairwise identifiers so the same natural person cannot be linked across trust boundaries.

Development

Elevated: **Non-Repudiation - PII or Auth Tokens Accessible in Browser-Side Storage: 1 / 1 Risk** - Exploitation likelihood is *Likely with Medium impact*.

Replace localStorage/sessionStorage token storage with HttpOnly, Secure, SameSite=Strict cookies. Cookies with the HttpOnly flag are inaccessible to JavaScript, eliminating the XSS token-theft vector. Deploy a strict Content-Security-Policy to limit damage from any residual XSS. Tag the asset has-httponly-session once deployed.

Operations

Elevated: **Detectability - PII Flows Without Audit Logging: 6 / 6 Risks - Exploitation likelihood is *Likely* with *Medium* impact.**

Set audit_logged to true on every communication link that transmits PII data assets. Store logs in a tamper-evident system separate from the application.

RAA Analysis

For each technical asset the **"Relative Attacker Attractiveness"** (RAA) value was calculated in percent. The higher the RAA, the more interesting it is for an attacker to compromise the asset. The calculation algorithm takes the sensitivity ratings and quantities of stored and processed data into account as well as the communication links of the technical asset. Neighbouring assets to high-value RAA targets might receive an increase in their RAA value when they have a communication link towards that target ("Pivoting-Factor").

The following lists all technical assets sorted by their RAA value from highest (most attacker attractive) to lowest. This list can be used to prioritize on efforts relevant for the most attacker-attractive technical assets:

Technical asset paragraphs are clickable and link to the corresponding chapter.

ECS Container Platform: RAA 100%

AWS ECS (Fargate) running the API and Nginx containers in production. INTENTIONAL: ECR Enhanced Scanning (Inspector v2) is not enabled. Running containers are not scanned post-deployment for newly disclosed CVEs. Container images are pulled from ECR without signature verification.

AWS RDS PostgreSQL: RAA 54%

AWS RDS PostgreSQL instance (production). Replaces the Docker-based PostgreSQL in production deployments. Automated backups enabled (7-day retention). INTENTIONAL: Encryption at rest is disabled (`storage_encrypted = false` in Terraform). Public accessibility is off but subnet group is in app subnet (not data subnet), creating potential exposure via compromised app instances.

PostgreSQL Database: RAA 54%

Primary relational datastore. Contains users table (email + bcrypt password hash), notes table (title + content), and attachments metadata. INTENTIONAL MISCONFIGURATION: No encryption at rest (encryption: none).

API Server: RAA 44%

Node.js/Express REST API. Handles authentication (JWT/bcrypt), CRUD on notes, file uploads to MinIO. Bridges the frontend and backend networks. Single most security-critical component — compromise equals full data access.

AWS API Gateway: RAA 30%

AWS API Gateway (HTTP API) fronting the ECS API containers in production. Routes traffic from the ALB to backend ECS tasks. INTENTIONAL: No usage plan or throttling policy configured. No WAF association on the API gateway stage. Auth relies entirely on the API server (JWT) — no gateway-level auth enforcement.

Browser SPA: RAA 29%

React Single Page Application served from Nginx. Users authenticate, manage notes, and upload files through this interface.

VaultNote Vector Store: RAA 26%

Qdrant vector database storing embedding vectors for semantic search and RAG context retrieval. Vectors are generated by the embedding service from user note content. INTENTIONAL: No encryption at rest configured (encryption: none). The vector store is queryable by the LLM pipeline and the API server without row-level isolation — all user embeddings share the same collection. No access control list separates per-user embedding queries.

Redis Cache: RAA 22%

In-memory store used as session store and JWT revocation list. INTENTIONAL MISCONFIGURATION: No encryption at rest. If Redis persistence is enabled and the RDB file is exfiltrated, session tokens are exposed.

AWS S3 Notes Bucket: RAA 21%

AWS S3 bucket storing file attachments for production deployments. Replaces MinIO S3-compatible storage in the cloud environment. Bucket is private (Block Public Access enabled) but server-side encryption is not configured. INTENTIONAL: SSE-S3 or SSE-KMS not enabled. Objects are stored unencrypted at the storage layer — provider-side access exposes raw data.

MinIO Object Storage: RAA 21%

S3-compatible object store for file attachments. INTENTIONAL MISCONFIGURATION: Uses default minioadmin/minioadmin credentials (visible in docker-compose.yml) and stores files without encryption at rest. HTTP-only (no TLS) within the Docker network.

LLM Note Summarizer: RAA 20%

OpenAI-compatible LLM inference endpoint used to generate AI summaries of user notes on demand. Deployed as a containerised model proxy (LiteLLM) with a public API endpoint for testing. INTENTIONAL: Endpoint is internet-accessible and lacks authentication — the has-authentication tag is intentionally absent to demonstrate the finding. No adversarial input shield (Llama Guard or guardrails) is configured. System prompt contains internal business context visible to adversarial probing.

GitHub Actions Build Pipeline: RAA 12%

GitHub Actions CI/CD pipeline. Builds Docker images, runs tests, pushes to ECR. INTENTIONAL: No SBOM generation (no Syft or CycloneDX step configured). No build provenance attestation (no SLSA provenance step). Hardcoded AWS_SECRET_ACCESS_KEY in workflow environment variables.

VaultNote Source Repository: RAA 12%

GitHub repository hosting the VaultNote monorepo (API, frontend, Terraform). Contains application secrets in compose files (minioadmin credentials visible in git history) and Terraform state references. INTENTIONAL: No branch protection rules configured — developers push directly to main, bypassing review and signed-commit requirements.

Note RAG Pipeline: RAA 11%

Retrieval-Augmented Generation pipeline that assembles context chunks from the vector store before forwarding to the LLM. Built on LangChain. INTENTIONAL: No input validation or injection guard on retrieved chunks (has-input-validation absent). A note containing prompt injection payloads could be retrieved as context and alter LLM behavior. No retrieval filtering — all results from the

vector store are injected into context without relevance threshold or content sanitisation.

AWS ECR Container Registry: RAA 9%

AWS Elastic Container Registry storing VaultNote Docker images (api, nginx). INTENTIONAL: Image vulnerability scanning is disabled (ECR Basic scanning only, not Enhanced/Inspector). Images are not signed — no Cosign/Sigstore policy enforced. Repository is private but no admission controller validates signatures at deploy time.

Lambda Email Notifier: RAA 2%

AWS Lambda function that sends transactional emails (note-share invitations, account verification) triggered by SQS messages from the API server. Processes user email addresses (PII) and note titles in notification payloads. INTENTIONAL: No dead letter queue (DLQ) configured on the SQS event source. Failed notifications are silently dropped after 3 retries with no audit trail.

Nginx Reverse Proxy: RAA 1%

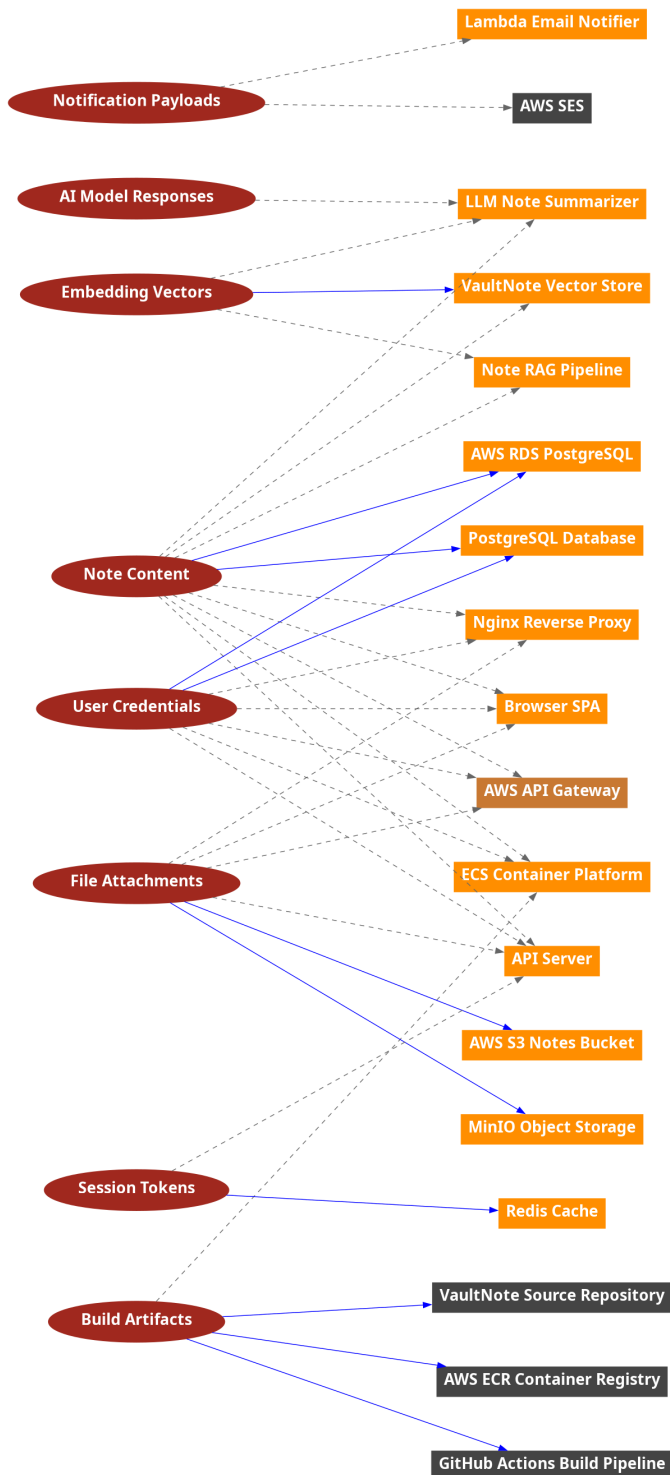
Nginx terminates TLS and forwards all API traffic to the Express server over plain HTTP (port 3000). This is the DMZ entry point.

AWS SES: RAA 0%

AWS Simple Email Service used for transactional email delivery (note-share invitations, account verification). Managed service — not custom-developed. SES sending identity (domain) is verified; DKIM and SPF records are configured. No dedicated sending quota alarm; a compromised Lambda could exhaust the daily send limit and cause notification delivery failure.

Data Mapping

The following diagram was generated by Threagile based on the model input and gives a high-level distribution of data assets across technical assets. The color matches the identified data breach probability and risk level (see the "Data Breach Probabilities" chapter for more details). A solid line stands for *data is stored by the asset* and a dashed one means *data is processed by the asset*. For a full high-resolution version of this diagram please refer to the PNG image file alongside this report.



Out-of-Scope Assets: 0 Assets

This chapter lists all technical assets that have been defined as out-of-scope. Each one should be checked in the model whether it should better be included in the overall risk analysis:

Technical asset paragraphs are clickable and link to the corresponding chapter.

No technical assets have been defined as out-of-scope.

Potential Model Failures: 0 / 0 Risks

This chapter lists potential model failures where not all relevant assets have been modeled or the model might itself contain inconsistencies. Each potential model failure should be checked in the model against the architecture design:

Risk finding paragraphs are clickable and link to the corresponding chapter.

No potential model failures have been identified.

Questions: 0 / 0 Questions

This chapter lists custom questions that arose during the threat modeling process.

No custom questions arose during the threat modeling process.

Identified Risks by Vulnerability category

In total **28 potential risks** have been identified during the threat modeling process of which **0 are rated as critical, 0 as high, 16 as elevated, 12 as medium, and 0 as low.**

These risks are distributed across **6 vulnerability categories**. The following sub-chapters of this section describe each identified risk category.

Data Disclosure - Personal Data Transmitted Over Unencrypted Channel: 2

Description (Information Disclosure): [CWE 319](#)

Personal data in transit over unencrypted communication links (plain HTTP, unencrypted database protocols, etc.) is exposed to network observers. GDPR Article 32 requires appropriate technical measures including encryption in transit.

Impact

Exposure of personal data to eavesdroppers, potential data breach notification obligations under GDPR Article 33, and significant regulatory penalties.

Detection Logic

In-scope technical assets that process PII data assets AND have at least one outgoing communication link that is unencrypted and not VPN-protected.

Risk Rating

High — unencrypted PII in transit is a direct GDPR Article 32 violation.

False Positives

Assets that process PII but only transmit it over TLS or VPN links; verify with the full threat model review.

Mitigation (Architecture): Encrypt all links that carry personal data

Migrate all communication links carrying PII data assets to TLS 1.2+ (preferably TLS 1.3). For internal service-to-service communication, consider mutual TLS (mTLS).

ASVS Chapter: [V9 - Communication Verification Requirements](#)

Cheat Sheet: [Transport Layer Security Cheat Sheet](#)

Check

Are all communication links carrying personal data encrypted in transit?

Risk Findings

The risk **Data Disclosure - Personal Data Transmitted Over Unencrypted Channel** was found **2 times** in the analyzed architecture to be potentially possible. Each spot should be checked individually by reviewing the implementation whether all controls have been applied properly in order to mitigate each risk.

Risk finding paragraphs are clickable and link to the corresponding chapter.

Elevated Risk Severity

Data Disclosure: API Server processes personal data and has an unencrypted outgoing communication link: Exploitation likelihood is *Likely with High* impact.

[data-disclosure-pii-unencrypted-link@api-server](#)

[In Progress](#) 2026-04-25 Security Team VAULT-117

The API server communicates with PostgreSQL, Redis, and MinIO over plaintext connections within backend-net. TLS for all internal database connections is planned post-launch in VAULT-117. MinIO credential rotation is tracked in VAULT-110 (blocker).

Data Disclosure: Nginx Reverse Proxy processes personal data and has an unencrypted outgoing communication link: Exploitation likelihood is *Likely with High* impact.

[data-disclosure-pii-unencrypted-link@nginx-proxy](#)

[In Progress](#) 2026-04-18 Security Team VAULT-102

Duplicate of VAULT-102 (Nginx.API plaintext link). Resolved when mTLS migration completes in sprint 22. LINDDUN data-disclosure finding is tracked under the same remediation work.

Detectability - PII Flows Without Audit Logging: 6 / 6 Risks

Description (Repudiation): [CWE 778](#)

Communication links that carry personal data without audit logging enabled mean that access to and use of personal data cannot be detected or reviewed after the fact. GDPR Article 5(2) requires demonstrable accountability.

Impact

Inability to detect unauthorized access to personal data, failure to meet GDPR accountability obligations, and potential inability to meet breach notification deadlines.

Detection Logic

In-scope technical assets that process PII AND have at least one outgoing link without audit logging enabled.

Risk Rating

Medium severity baseline; elevated if the data includes sensitive categories.

False Positives

Fully tokenised flows, or flows covered by audit logging at a gateway layer.

Mitigation (Operations): Enable audit logging on all links carrying personal data

Set `audit_logged` to true on every communication link that transmits PII data assets. Store logs in a tamper-evident system separate from the application.

ASVS Chapter: [V7 - Error Handling and Logging Verification Requirements](#)

Cheat Sheet: [Logging Cheat Sheet](#)

Check

Are all communication links that carry personal data audit-logged?

Risk Findings

The risk **Detectability - PII Flows Without Audit Logging** was found **6 times** in the analyzed architecture to be potentially possible. Each spot should be checked individually by reviewing the implementation whether all controls have been applied properly in order to mitigate each risk.

Risk finding paragraphs are clickable and link to the corresponding chapter.

Elevated Risk Severity

Detectability: ECS Container Platform processes PII and has communication links without audit logging: Exploitation likelihood is *Likely* with *Medium* impact.

[detecting-no-audit-logging@ecs-platform](#)

Unchecked

Detectability: LLM Note Summarizer processes PII and has communication links without audit logging: Exploitation likelihood is *Likely* with *Medium* impact.

[detecting-no-audit-logging@llm-summarizer](#)

Unchecked

Detectability: Lambda Email Notifier processes PII and has communication links without audit logging: Exploitation likelihood is *Likely* with *Medium* impact.

[detecting-no-audit-logging@lambda-notifier](#)

Unchecked

Detectability: Note RAG Pipeline processes PII and has communication links without audit logging: Exploitation likelihood is *Likely* with *Medium* impact.

[detecting-no-audit-logging@rag-pipeline](#)

Unchecked

Detectability: Nginx Reverse Proxy processes PII and has communication links without audit logging: Exploitation likelihood is *Likely* with *Medium* impact.

[detecting-no-audit-logging@nginx-proxy](#)

Accepted

2026-04-25

Security Team

VAULT-115

Nginx access.log captures all incoming requests (IP, URL, status code, response size). Individual API-level audit events are handled by the API server. The Nginx.API internal hop has no separate audit log; this is accepted as the TLS-termination boundary log is sufficient for DMZ-level audit requirements.

Detectability: API Server processes PII and has communication links without audit logging: Exploitation likelihood is *Likely* with *Medium* impact.

[detecting-no-audit-logging@api-server](#)

In Progress

2026-04-25

Security Team

VAULT-116

Structured request/response audit logging (Morgan + Winston) is implemented for authenticated endpoints but coverage is incomplete for internal data-access events. Full audit trail covering all PII-processing operations is tracked in VAULT-116. Target: sprint 23.

Identifiability - Personal Data Stored Without Encryption at Rest: 6 / 6 Risk

Description (Information Disclosure): [CWE 312](#)

Datastores holding personal data without encryption at rest allow anyone with access to the underlying storage to read and identify data subjects. GDPR Article 32 requires appropriate security measures including encryption at rest.

Impact

Physical or logical access to unencrypted volumes directly exposes personal data, enabling identification of natural persons and constituting a reportable data breach.

Detection Logic

In-scope datastore technical assets storing at least one PII data asset with the encryption field set to none.

Risk Rating

Severity matches the confidentiality classification of the PII stored. Strictly confidential PII warrants Critical.

False Positives

Datastores on a host protected by full-disk encryption with a documented configuration; document and accept with justification.

Mitigation (Architecture): Enable encryption at rest on all datastores holding personal data

Enable disk-level or database-level encryption on every datastore asset that stores PII data assets. Use full-disk encryption (LUKS, AWS EBS encryption, Azure SSE) or database-level TDE (pgcrypto, SQL Server TDE).

ASVS Chapter: [V6 - Stored Cryptography Verification Requirements](#)

Cheat Sheet: [Cryptographic_Storage_Cheat_Sheet](#)

Check

Is every datastore that holds personal data encrypted at rest?

Risk Findings

The risk **Identifiability - Personal Data Stored Without Encryption at Rest** was found **6 times** in the analyzed architecture to be potentially possible. Each spot should be checked individually by reviewing the implementation whether all controls have been applied properly in order to mitigate each risk.

Risk finding paragraphs are clickable and link to the corresponding chapter.

Elevated Risk Severity

Identifiability: datastore **AWS RDS PostgreSQL** stores personal data without encryption at rest: Exploitation likelihood is *Likely with High* impact.

[identifying-pii-stored-unencrypted@rds-postgresql](#)

Unchecked

Identifiability: datastore **AWS S3 Notes Bucket** stores personal data without encryption at rest: Exploitation likelihood is *Likely with High* impact.

[identifying-pii-stored-unencrypted@s3-notes-bucket](#)

Unchecked

Identifiability: datastore **MinIO Object Storage** stores personal data without encryption at rest: Exploitation likelihood is *Likely with High* impact.

[identifying-pii-stored-unencrypted@minio-storage](#)

Unchecked

Identifiability: datastore **PostgreSQL Database** stores personal data without encryption at rest: Exploitation likelihood is *Likely with High* impact.

[identifying-pii-stored-unencrypted@postgresql-db](#)

Unchecked

Identifiability: datastore **Redis Cache** stores personal data without encryption at rest: Exploitation likelihood is *Likely with High* impact.

[identifying-pii-stored-unencrypted@redis-cache](#)

Unchecked

Identifiability: datastore **VaultNote Vector Store** stores personal data without encryption at rest: Exploitation likelihood is *Likely with High* impact.

[identifying-pii-stored-unencrypted@vector-store](#)

Unchecked

Non-Repudiation - PII or Auth Tokens Accessible in Browser-Side Storage

Description (Information Disclosure): [CWE 922](#)

Browser-based clients (SPAs, web apps) that process personal data or authentication tokens typically persist them in `localStorage` or `sessionStorage`. This browser-accessible storage is readable by any JavaScript running in the same origin â€” including injected scripts. Without `HttpOnly` cookies or a strict Content-Security-Policy, the stored tokens and PII are one XSS payload away from full exfiltration. From a LINDDUN perspective this enables Non-Repudiation attacks: an adversary who steals a token can act as the victim, making it impossible to distinguish attacker actions from legitimate user actions in audit logs.

Impact

Stolen session tokens allow account takeover and impersonation. The original user cannot repudiate actions performed by the attacker. Stolen PII violates GDPR data minimisation and storage limitation principles.

Detection Logic

In-scope technical assets with browser (client) technology that process PII data assets and are not tagged with `has-httponly-session`.

Risk Rating

Medium baseline. Elevated when tokens have long TTL (>1 hour) or when no CSP is configured on the serving origin.

False Positives

Applications that exclusively use `HttpOnly` cookies for session management and do not store any PII in JavaScript-accessible storage. Tag the asset `has-httponly-session` to suppress this finding once verified.

Mitigation (Development): Move session tokens to `HttpOnly` cookies; apply Content-Security-Policy

Replace `localStorage/sessionStorage` token storage with `HttpOnly`, `Secure`, `SameSite=Strict` cookies. Cookies with the `HttpOnly` flag are inaccessible to JavaScript, eliminating the XSS token-theft vector. Deploy a strict Content-Security-Policy to limit damage from any residual XSS. Tag the asset `has-httponly-session` once deployed.

ASVS Chapter: [V3 - Session Management Verification Requirements](#)
Cheat Sheet: [HTML5 Security Cheat Sheet](#)

Check

Are authentication tokens stored in HttpOnly cookies rather than localStorage? Is a Content-Security-Policy header present on all pages served to this browser client?

Risk Findings

The risk **Non-Repudiation - PII or Auth Tokens Accessible in Browser-Side Storage** was found **1 time** in the analyzed architecture to be potentially possible. Each spot should be checked individually by reviewing the implementation whether all controls have been applied properly in order to mitigate each risk.

Risk finding paragraphs are clickable and link to the corresponding chapter.

Elevated Risk Severity

PII in Client-Side Storage: browser asset **Browser SPA** processes personal data that may be stored in JavaScript-accessible browser storage: Exploitation likelihood is *Likely* with *Medium* impact.

[pii-client-side-storage@browser-spa](#)

Unchecked

Unawareness - PII Processed Without a Consent Management Component

Description (Information Disclosure): [CWE 359](#)

When a system processes personal data from end users but has no consent-management technology in the architecture, data subjects are likely unaware of the nature and extent of processing. GDPR Articles 13 and 14 require transparent disclosure.

Impact

Data subjects cannot exercise GDPR rights. Regulatory investigation risk and class-action exposure in jurisdictions with private right of action.

Detection Logic

In-scope user-facing technical assets that process PII data assets where no other asset carries a consent_collector technology attribute.

Risk Rating

Medium baseline; elevated for consumer-facing applications or special-category data.

False Positives

B2B systems under a contract lawful basis where consent is not applicable.

Mitigation (Architecture): Add a consent management technical asset to the architecture

Introduce a consent-manager (tagged with technology consent-manager) to capture, store, and honour data-subject preferences.

ASVS Chapter: [V8 - Data Protection Verification Requirements](#)

Cheat Sheet: [Logging_Cheat_Sheet](#)

Check

Does the architecture include a consent management component when processing PII?

Risk Findings

The risk **Unawareness - PII Processed Without a Consent Management Component** was found **1 time** in the analyzed architecture to be potentially possible. Each spot should be checked individually by reviewing the implementation whether all controls have been applied properly in order to mitigate each risk.

Risk finding paragraphs are clickable and link to the corresponding chapter.

Elevated Risk Severity

Unawareness: Browser SPA processes end-user PII but no consent management component is present: Exploitation likelihood is *Likely* with *Medium* impact.

[unawareness-no-consent-technology@browser-spa](#)

[In Progress](#)

2026-04-25

Privacy Team

VAULT-118

A consent management platform (CMP) is being evaluated (OneTrust / Osano). In the interim, the privacy policy is displayed at registration and users explicitly accept terms before account creation. Formal CMP integration is planned for Q3 2026.

Linkability - Same PII Data Asset Processed Across Multiple Trust Boundaries

Description (Information Disclosure): [CWE 359](#)

When the same personal-data asset is processed by technical assets in two or more different trust boundaries, an attacker who compromises any one boundary gains linkability across all others, enabling cross-context profiling.

Impact

Aggregation of personal data enables profiling, re-identification, and violations of GDPR data minimisation and purpose limitation principles (Articles 5(1)(b) and 5(1)(c)).

Detection Logic

Technical assets that process PII data assets that also appear in assets in at least one other trust boundary.

Risk Rating

Medium baseline; elevated for sensitive categories or high-quantity data.

False Positives

Internal service meshes within a single logical zone modelled as separate trust boundaries.

Mitigation (Architecture): Apply data minimisation and per-context pseudonymisation

Use per-context pseudonyms or pairwise identifiers so the same natural person cannot be linked across trust boundaries.

ASVS Chapter: [V8 - Data Protection Verification Requirements](#)

Cheat Sheet: [Attack_Surface_Analysis_Cheat_Sheet](#)

Check

Does each PII data asset appear in assets across more than one trust boundary?

Risk Findings

The risk **Linkability - Same PII Data Asset Processed Across Multiple Trust Boundaries** was found **12 times** in the analyzed architecture to be potentially possible. Each spot should be checked individually by reviewing the implementation whether all controls have been applied properly in order to mitigate each risk.

Risk finding paragraphs are clickable and link to the corresponding chapter.

Medium Risk Severity

Linkability: AWS API Gateway processes a PII data asset also flowing into another trust boundary: Exploitation likelihood is *Unlikely* with *Medium* impact.

[linking-pii-multi-boundary@aws-api-gateway](#)

Unchecked

Linkability: AWS RDS PostgreSQL processes a PII data asset also flowing into another trust boundary: Exploitation likelihood is *Unlikely* with *Medium* impact.

[linking-pii-multi-boundary@rds-postgresql](#)

Unchecked

Linkability: AWS S3 Notes Bucket processes a PII data asset also flowing into another trust boundary: Exploitation likelihood is *Unlikely* with *Medium* impact.

[linking-pii-multi-boundary@s3-notes-bucket](#)

Unchecked

Linkability: ECS Container Platform processes a PII data asset also flowing into another trust boundary: Exploitation likelihood is *Unlikely* with *Medium* impact.

[linking-pii-multi-boundary@ecs-platform](#)

Unchecked

Linkability: Note RAG Pipeline processes a PII data asset also flowing into another trust boundary: Exploitation likelihood is *Unlikely* with *Medium* impact.

[linking-pii-multi-boundary@rag-pipeline](#)

Unchecked

Linkability: VaultNote Vector Store processes a PII data asset also flowing into another trust boundary: Exploitation likelihood is *Unlikely* with *Medium* impact.

[linking-pii-multi-boundary@vector-store](#)

Unchecked

Linkability: API Server processes a PII data asset also flowing into another trust boundary: Exploitation likelihood is *Unlikely* with *Medium* impact.

[linking-pii-multi-boundary@api-server](#)

Accepted

2026-04-25 Security Team

VAULT-119

The API server bridges frontend-net and backend-net by architectural necessity — it is the sole legitimate data path between the two networks. Cross-boundary PII flow is an inherent design characteristic, not a misconfiguration.

Mitigated by authentication + authorization enforcement at the API layer.

Linkability: Browser SPA processes a PII data asset also flowing into another trust boundary: Exploitation likelihood is *Unlikely* with *Medium* impact.

linking-pii-multi-boundary@browser-spa

Accepted 2026-04-25 Security Team VAULT-119

The Browser SPA processes user-credentials and note-content across the Internet and DMZ boundaries by design (user-facing client). The risk is inherent to the SPA architecture; mitigated by HTTPS-only transport and short-lived session tokens.

Linkability: MinIO Object Storage processes a PII data asset also flowing into another trust boundary: Exploitation likelihood is *Unlikely* with *Medium* impact.

linking-pii-multi-boundary@minio-storage

Accepted 2026-04-25 Security Team VAULT-119

File attachments uploaded via the Internet are stored in the Data Tier. Cross-boundary flow is by design. VAULT-110 (credential rotation + SSE) addresses the high-severity storage risk; the linkability finding is accepted as an inherent architectural property.

Linkability: Nginx Reverse Proxy processes a PII data asset also flowing into another trust boundary: Exploitation likelihood is *Unlikely* with *Medium* impact.

linking-pii-multi-boundary@nginx-proxy

Accepted 2026-04-25 Security Team VAULT-119

Nginx is the TLS-termination proxy and must bridge Internet and DMZ by design. Cross-boundary PII flow is intentional and unavoidable for this component.

Linkability: PostgreSQL Database processes a PII data asset also flowing into another trust boundary: Exploitation likelihood is *Unlikely* with *Medium* impact.

linking-pii-multi-boundary@postgresql-db

Accepted 2026-04-25 Security Team VAULT-119

PostgreSQL stores credentials and note content that originate from multiple trust boundaries (Internet . DMZ . App Network . Data Tier). This is inherent to the data flow by design. Risk is accepted; mitigated by parameterized queries and authentication-only access via the API.

Linkability: Redis Cache processes a PII data asset also flowing into another trust boundary: Exploitation likelihood is *Unlikely* with *Medium* impact.

linking-pii-multi-boundary@redis-cache

Accepted 2026-04-25 Security Team VAULT-119

Session identifiers originate from users across the Internet boundary. Storage in Redis within the Data Tier is unavoidable. Mitigated by short TTL (15 minutes), revocation list, and password-protected access.

Identified Risks by Technical Asset

In total **28 potential risks** have been identified during the threat modeling process of which **0 are rated as critical, 0 as high, 16 as elevated, 12 as medium, and 0 as low.**

These risks are distributed across **18 in-scope technical assets**. The following sub-chapters of this section describe each identified risk grouped by technical asset. The RAA value of a technical asset is the calculated "Relative Attacker Attractiveness" value in percent.

API Server: 3 / 3 Risks

Description

Node.js/Express REST API. Handles authentication (JWT/bcrypt), CRUD on notes, file uploads to MinIO. Bridges the frontend and backend networks. Single most security-critical component — compromise equals full data access.

Identified Risks of Asset

Risk finding paragraphs are clickable and link to the corresponding chapter.

Elevated Risk Severity

Data Disclosure: API Server processes personal data and has an unencrypted outgoing communication link: Exploitation likelihood is *Likely* with *High* impact.

[data-disclosure-pii-unencrypted-link@api-server](#)

[In Progress](#)

2026-04-25

Security Team

VAULT-117

The API server communicates with PostgreSQL, Redis, and MinIO over plaintext connections within backend-net. TLS for all internal database connections is planned post-launch in VAULT-117. MinIO credential rotation is tracked in VAULT-110 (blocker).

Detectability: API Server processes PII and has communication links without audit logging: Exploitation likelihood is *Likely* with *Medium* impact.

[detecting-no-audit-logging@api-server](#)

[In Progress](#)

2026-04-25

Security Team

VAULT-116

Structured request/response audit logging (Morgan + Winston) is implemented for authenticated endpoints but coverage is incomplete for internal data-access events. Full audit trail covering all PII-processing operations is tracked in VAULT-116. Target: sprint 23.

Medium Risk Severity

Linkability: API Server processes a PII data asset also flowing into another trust boundary: Exploitation likelihood is *Unlikely* with *Medium* impact.

[linking-pii-multi-boundary@api-server](#)

[Accepted](#)

2026-04-25

Security Team

VAULT-119

The API server bridges frontend-net and backend-net by architectural necessity — it is the sole legitimate data path between the two networks. Cross-boundary PII flow is an inherent design characteristic, not a misconfiguration. Mitigated by authentication + authorization enforcement at the API layer.

Asset Information

ID:	api-server
Type:	process
Usage:	business
RAA:	44 %

Size:	service
Technology:	web-service-rest
Tags:	express, jwt, nodejs, trike-trust-level-admin
Internet:	false
Machine:	container
Encryption:	none
Multi-Tenant:	false
Redundant:	false
Custom-Developed:	true
Client by Human:	false
Data Processed:	File Attachments, Note Content, Session Tokens, User Credentials
Data Stored:	none
Formats Accepted:	JSON

Asset Rating

Owner:	VaultNote Team
Confidentiality:	strictly-confidential (rated 5 in scale of 5)
Integrity:	critical (rated 4 in scale of 5)
Availability:	critical (rated 4 in scale of 5)
CIA-Justification:	The API processes all user data. It bridges both Docker networks — a compromised API container can pivot into the internal data tier.

Outgoing Communication Links: 3

Target technical asset names are clickable and link to the corresponding chapter.

Redis Session Store (outgoing)

Session token storage and JWT revocation list. Password-protected (requirepass) but no TLS.

Target:	Redis Cache
Protocol:	nosql-access-protocol
Encrypted:	false
Authentication:	credentials
Authorization:	technical-user
Read-Only:	false
Usage:	business
Tags:	none
VPN:	false

IP-Filtered: false
Data Sent: Session Tokens
Data Received: Session Tokens

PostgreSQL Connection (outgoing)

Application-level SQL queries for user auth, notes CRUD. Uses parameterized queries (pg library) to prevent SQL injection. No TLS on the connection — traffic is plaintext within backend-net.

Target: PostgreSQL Database
Protocol: sql-access-protocol
Encrypted: false
Authentication: credentials
Authorization: technical-user
Read-Only: false
Usage: business
Tags: none
VPN: false
IP-Filtered: false
Data Sent: Note Content, User Credentials
Data Received: Note Content, User Credentials

MinIO File Storage (outgoing)

INTENTIONAL MISCONFIGURATION: S3 API calls to MinIO over plain HTTP. Default minioadmin/minioadmin credentials are hardcoded in compose. Any container on backend-net can read all stored files.

Target: MinIO Object Storage
Protocol: http
Encrypted: false
Authentication: credentials
Authorization: technical-user
Read-Only: false
Usage: business
Tags: intentional-misconfiguration
VPN: false
IP-Filtered: false
Data Sent: File Attachments
Data Received: File Attachments

Incoming Communication Links: 2

Source technical asset names are clickable and link to the corresponding chapter.

HTTP to API Server (incoming)

INTENTIONAL MISCONFIGURATION: Traffic between Nginx and the API server uses plaintext HTTP. An attacker with access to the Docker network (container escape, compromised sidecar) can intercept credentials, session tokens, and note content in transit.

Source:	Nginx Reverse Proxy
Protocol:	http
Encrypted:	false
Authentication:	none
Authorization:	none
Read-Only:	false
Usage:	business
Tags:	intentional-misconfiguration
VPN:	false
IP-Filtered:	false
Data Received:	File Attachments, Note Content, User Credentials
Data Sent:	File Attachments, Note Content

Route to ECS API (incoming)

HTTP forward from API Gateway to the ECS-hosted API containers via ALB. Traffic is within the AWS VPC — no public routing for backend traffic.

Source:	AWS API Gateway
Protocol:	https
Encrypted:	true
Authentication:	none
Authorization:	none
Read-Only:	false
Usage:	business
Tags:	none
VPN:	false
IP-Filtered:	true
Data Received:	File Attachments, Note Content, User Credentials
Data Sent:	File Attachments, Note Content

AWS RDS PostgreSQL: 2 / 2 Risks

Description

AWS RDS PostgreSQL instance (production). Replaces the Docker-based PostgreSQL in production deployments. Automated backups enabled (7-day retention). INTENTIONAL: Encryption at rest is disabled (`storage_encrypted = false` in Terraform). Public accessibility is off but subnet group is in app subnet (not data subnet), creating potential exposure via compromised app instances.

Identified Risks of Asset

Risk finding paragraphs are clickable and link to the corresponding chapter.

Elevated Risk Severity

Identifiability: datastore **AWS RDS PostgreSQL** stores personal data without encryption at rest: Exploitation likelihood is *Likely* with *High* impact.

[identifying-pii-stored-unencrypted@rds-postgresql](#)

Unchecked

Medium Risk Severity

Linkability: **AWS RDS PostgreSQL** processes a PII data asset also flowing into another trust boundary: Exploitation likelihood is *Unlikely* with *Medium* impact.

[linking-pii-multi-boundary@rds-postgresql](#)

Unchecked

Asset Information

ID:	rds-postgresql
Type:	datastore
Usage:	business
RAA:	54 %
Size:	service
Technology:	managed-database
Tags:	aws, cloud-native, criticality-high, has-automated-backup, information-asset-container, postgresql, rds
Internet:	false
Machine:	virtual
Encryption:	none
Multi-Tenant:	false

Redundant: true
Custom-Developed: false
Client by Human: false
Data Processed: Note Content, User Credentials
Data Stored: Note Content, User Credentials
Formats Accepted: JSON

Asset Rating

Owner: VaultNote Team
Confidentiality: strictly-confidential (rated 5 in scale of 5)
Integrity: critical (rated 4 in scale of 5)
Availability: critical (rated 4 in scale of 5)
CIA-Justification: Production database stores all user PII and note content. Encryption disabled means RDS snapshot exfiltration exposes all data in plaintext. Snapshot sharing misconfiguration (a common AWS incident) bypasses all access controls.

AWS S3 Notes Bucket: 2 / 2 Risks

Description

AWS S3 bucket storing file attachments for production deployments. Replaces MinIO S3-compatible storage in the cloud environment. Bucket is private (Block Public Access enabled) but server-side encryption is not configured. INTENTIONAL: SSE-S3 or SSE-KMS not enabled. Objects are stored unencrypted at the storage layer — provider-side access exposes raw data.

Identified Risks of Asset

Risk finding paragraphs are clickable and link to the corresponding chapter.

Elevated Risk Severity

Identifiability: datastore **AWS S3 Notes Bucket** stores personal data without encryption at rest: Exploitation likelihood is *Likely* with *High* impact.

[identifying-pii-stored-unencrypted@s3-notes-bucket](#)

Unchecked

Medium Risk Severity

Linkability: **AWS S3 Notes Bucket** processes a PII data asset also flowing into another trust boundary: Exploitation likelihood is *Unlikely* with *Medium* impact.

[linking-pii-multi-boundary@s3-notes-bucket](#)

Unchecked

Asset Information

ID:	s3-notes-bucket
Type:	datastore
Usage:	business
RAA:	21 %
Size:	service
Technology:	object-storage
Tags:	aws, cloud-native, criticality-high, information-asset-container, s3, third-party-shared
Internet:	false
Machine:	virtual
Encryption:	none
Multi-Tenant:	false
Redundant:	true

Custom-Developed: false
Client by Human: false
Data Processed: File Attachments
Data Stored: File Attachments
Formats Accepted: File

Asset Rating

Owner: VaultNote Team
Confidentiality: confidential (rated 4 in scale of 5)
Integrity: critical (rated 4 in scale of 5)
Availability: operational (rated 2 in scale of 5)
CIA-Justification: S3 stores user-uploaded files which may contain sensitive documents. No SSE means provider-side access (support channel, storage engineer) reads raw data. third-party-shared because S3 is operated by AWS and subject to subprocessor data protection requirements. No DPA review completed.

Browser SPA: 3 / 3 Risks

Description

React Single Page Application served from Nginx. Users authenticate, manage notes, and upload files through this interface.

Identified Risks of Asset

Risk finding paragraphs are clickable and link to the corresponding chapter.

Elevated Risk Severity

PII in Client-Side Storage: browser asset **Browser SPA** processes personal data that may be stored in JavaScript-accessible browser storage: Exploitation likelihood is *Likely* with *Medium* impact.

[pii-client-side-storage@browser-spa](#)

Unchecked

Unawareness: Browser SPA processes end-user PII but no consent management component is present: Exploitation likelihood is *Likely* with *Medium* impact.

[unawareness-no-consent-technology@browser-spa](#)

In Progress

2026-04-25

Privacy Team

VAULT-118

A consent management platform (CMP) is being evaluated (OneTrust / Osano). In the interim, the privacy policy is displayed at registration and users explicitly accept terms before account creation. Formal CMP integration is planned for Q3 2026.

Medium Risk Severity

Linkability: Browser SPA processes a PII data asset also flowing into another trust boundary: Exploitation likelihood is *Unlikely* with *Medium* impact.

[linking-pii-multi-boundary@browser-spa](#)

Accepted

2026-04-25

Security Team

VAULT-119

The Browser SPA processes user-credentials and note-content across the Internet and DMZ boundaries by design (user-facing client). The risk is inherent to the SPA architecture; mitigated by HTTPS-only transport and short-lived session tokens.

Asset Information

ID:	browser-spa
Type:	external-entity
Usage:	business
RAA:	29 %
Size:	application
Technology:	browser

Tags:	react, spa, trike-actor, trike-actor-untrusted
Internet:	true
Machine:	physical
Encryption:	none
Multi-Tenant:	false
Redundant:	false
Custom-Developed:	true
Client by Human:	true
Data Processed:	File Attachments, Note Content, User Credentials
Data Stored:	none
Formats Accepted:	JSON

Asset Rating

Owner:	End Users
Confidentiality:	strictly-confidential (rated 5 in scale of 5)
Integrity:	critical (rated 4 in scale of 5)
Availability:	operational (rated 2 in scale of 5)
CIA-Justification:	Client-side code is public; the trust boundary here is user data entered into the browser. XSS would allow an attacker to exfiltrate session tokens.

Outgoing Communication Links: 1

Target technical asset names are clickable and link to the corresponding chapter.

HTTPS to Nginx (outgoing)

User-initiated HTTPS requests carrying credentials (login) and note content (create/edit). This is the only encrypted link in the chain.

Target:	Nginx Reverse Proxy
Protocol:	https
Encrypted:	true
Authentication:	credentials
Authorization:	end-user-identity-propagation
Read-Only:	false
Usage:	business
Tags:	none
VPN:	false
IP-Filtered:	false

Data Sent: File Attachments, Note Content, User Credentials

Data Received: File Attachments, Note Content

ECS Container Platform: 2 / 2 Risks

Description

AWS ECS (Fargate) running the API and Nginx containers in production. INTENTIONAL: ECR Enhanced Scanning (Inspector v2) is not enabled. Running containers are not scanned post-deployment for newly disclosed CVEs. Container images are pulled from ECR without signature verification.

Identified Risks of Asset

Risk finding paragraphs are clickable and link to the corresponding chapter.

Elevated Risk Severity

Detectability: ECS Container Platform processes PII and has communication links without audit logging: Exploitation likelihood is *Likely* with *Medium* impact.

[detecting-no-audit-logging@ecs-platform](#)

Unchecked

Medium Risk Severity

Linkability: ECS Container Platform processes a PII data asset also flowing into another trust boundary: Exploitation likelihood is *Unlikely* with *Medium* impact.

[linking-pii-multi-boundary@ecs-platform](#)

Unchecked

Asset Information

ID:	ecs-platform
Type:	process
Usage:	business
RAA:	100 %
Size:	service
Technology:	container-platform
Tags:	aws, cloud-native, ecs, fargate
Internet:	false
Machine:	virtual
Encryption:	transparent
Multi-Tenant:	false
Redundant:	true
Custom-Developed:	false

Client by Human: false
Data Processed: Build Artifacts, Note Content, User Credentials
Data Stored: none
Formats Accepted: JSON

Asset Rating

Owner: VaultNote Team
Confidentiality: strictly-confidential (rated 5 in scale of 5)
Integrity: critical (rated 4 in scale of 5)
Availability: critical (rated 4 in scale of 5)
CIA-Justification: ECS runs the production API containers. Container escape or image tampering grants access to the application runtime and its mounted secrets. No image scanning means known CVEs in base images are undetected until exploitation.

Outgoing Communication Links: 1

Target technical asset names are clickable and link to the corresponding chapter.

Pull Images from ECR (outgoing)

ECS task pulls Docker images from ECR at task launch. Images are not verified against signatures before execution.

Target: AWS ECR Container Registry
Protocol: https
Encrypted: true
Authentication: credentials
Authorization: technical-user
Read-Only: true
Usage: devops
Tags: none
VPN: false
IP-Filtered: true
Data Sent: none
Data Received: Build Artifacts

LLM Note Summarizer: 1 / 1 Risk

Description

OpenAI-compatible LLM inference endpoint used to generate AI summaries of user notes on demand. Deployed as a containerised model proxy (LiteLLM) with a public API endpoint for testing. **INTENTIONAL:** Endpoint is internet-accessible and lacks authentication — the has-authentication tag is intentionally absent to demonstrate the finding. No adversarial input shield (Llama Guard or guardrails) is configured. System prompt contains internal business context visible to adversarial probing.

Identified Risks of Asset

Risk finding paragraphs are clickable and link to the corresponding chapter.

Elevated Risk Severity

Detectability: LLM Note Summarizer processes PII and has communication links without audit logging: Exploitation likelihood is *Likely* with *Medium* impact.

[detecting-no-audit-logging@llm-summarizer](#)

Unchecked

Asset Information

ID:	llm-summarizer
Type:	process
Usage:	business
RAA:	20 %
Size:	service
Technology:	llm-inference-endpoint
Tags:	ai, cloud-native, llm, openai
Internet:	true
Machine:	virtual
Encryption:	transparent
Multi-Tenant:	false
Redundant:	false
Custom-Developed:	true
Client by Human:	false
Data Processed:	AI Model Responses, Embedding Vectors, Note Content
Data Stored:	none
Formats Accepted:	JSON

Asset Rating

Owner:	VaultNote Team	
Confidentiality:	confidential	(rated 4 in scale of 5)
Integrity:	critical	(rated 4 in scale of 5)
Availability:	operational	(rated 2 in scale of 5)
CIA-Justification:	LLM endpoint processes user note content. Unauthenticated internet exposure allows arbitrary prompt submission, system prompt extraction via jailbreaks, and inference cost abuse. No guardrails means malicious inputs could generate harmful content or exfiltrate context window data.	

Outgoing Communication Links: 2

Target technical asset names are clickable and link to the corresponding chapter.

Query Vector Store (outgoing)

LLM retrieves relevant note chunks from the vector store to build RAG context. Similarity search over user-owned embeddings.

Target:	VaultNote Vector Store
Protocol:	https
Encrypted:	true
Authentication:	credentials
Authorization:	technical-user
Read-Only:	true
Usage:	business
Tags:	none
VPN:	false
IP-Filtered:	false
Data Sent:	Embedding Vectors
Data Received:	Note Content

Call RAG Pipeline (outgoing)

LLM inference endpoint triggers the RAG pipeline to build context from stored note embeddings before generating a summary response.

Target:	Note RAG Pipeline
Protocol:	https
Encrypted:	true
Authentication:	credentials

Authorization: technical-user
Read-Only: false
Usage: business
Tags: none
VPN: false
IP-Filtered: false
Data Sent: Note Content
Data Received: Note Content

Lambda Email Notifier: 1 / 1 Risk

Description

AWS Lambda function that sends transactional emails (note-share invitations, account verification) triggered by SQS messages from the API server. Processes user email addresses (PII) and note titles in notification payloads. INTENTIONAL: No dead letter queue (DLQ) configured on the SQS event source. Failed notifications are silently dropped after 3 retries with no audit trail.

Identified Risks of Asset

Risk finding paragraphs are clickable and link to the corresponding chapter.

Elevated Risk Severity

Detectability: **Lambda Email Notifier** processes PII and has communication links without audit logging: Exploitation likelihood is *Likely* with *Medium* impact.

[detecting-no-audit-logging@lambda-notifier](#)

Unchecked

Asset Information

ID:	lambda-notifier
Type:	process
Usage:	business
RAA:	2 %
Size:	component
Technology:	serverless-function
Tags:	aws, cloud-native, lambda, notifications
Internet:	false
Machine:	virtual
Encryption:	transparent
Multi-Tenant:	false
Redundant:	false
Custom-Developed:	true
Client by Human:	false
Data Processed:	Notification Payloads
Data Stored:	none
Formats Accepted:	JSON

Asset Rating

Owner:	VaultNote Team	
Confidentiality:	confidential	(rated 4 in scale of 5)
Integrity:	operational	(rated 2 in scale of 5)
Availability:	operational	(rated 2 in scale of 5)
CIA-Justification:	Lambda processes email addresses and note titles. PII loss due to silent failure creates compliance issues (incomplete processing records). Overpermissive IAM role could be exploited to access other AWS resources.	

Outgoing Communication Links: 1

Target technical asset names are clickable and link to the corresponding chapter.

SES Email Send (outgoing)

Lambda calls AWS SES to deliver transactional emails. IAM role grants ses:SendEmail only; no SendRawEmail permission. No SES sending rate limit configured on the IAM policy — a compromised Lambda could exhaust the SES daily quota.

Target:	AWS SES
Protocol:	https
Encrypted:	true
Authentication:	credentials
Authorization:	technical-user
Read-Only:	false
Usage:	business
Tags:	none
VPN:	false
IP-Filtered:	false
Data Sent:	Notification Payloads
Data Received:	none

MinIO Object Storage: 2 / 2 Risks

Description

S3-compatible object store for file attachments. INTENTIONAL MISCONFIGURATION: Uses default minioadmin/minioadmin credentials (visible in docker-compose.yml) and stores files without encryption at rest. HTTP-only (no TLS) within the Docker network.

Identified Risks of Asset

Risk finding paragraphs are clickable and link to the corresponding chapter.

Elevated Risk Severity

Identifiability: datastore **MinIO Object Storage** stores personal data without encryption at rest: Exploitation likelihood is *Likely* with *High* impact.

[identifying-pii-stored-unencrypted@minio-storage](#)

Unchecked

Medium Risk Severity

Linkability: **MinIO Object Storage** processes a PII data asset also flowing into another trust boundary: Exploitation likelihood is *Unlikely* with *Medium* impact.

[linking-pii-multi-boundary@minio-storage](#)

Accepted

2026-04-25 Security Team

VAULT-119

File attachments uploaded via the Internet are stored in the Data Tier. Cross-boundary flow is by design. VAULT-110 (credential rotation + SSE) addresses the high-severity storage risk; the linkability finding is accepted as an inherent architectural property.

Asset Information

ID:	minio-storage
Type:	datastore
Usage:	business
RAA:	21 %
Size:	service
Technology:	object-storage
Tags:	criticality-high, information-asset-container, intentional-misconfiguration, minio, object-storage, s3, third-party-shared
Internet:	false
Machine:	container
Encryption:	none
Multi-Tenant:	false

Redundant: false
Custom-Developed: false
Client by Human: false
Data Processed: File Attachments
Data Stored: File Attachments
Formats Accepted: File

Asset Rating

Owner: VaultNote Team
Confidentiality: confidential (rated 4 in scale of 5)
Integrity: critical (rated 4 in scale of 5)
Availability: operational (rated 2 in scale of 5)
CIA-Justification: Stores user file uploads which may contain sensitive documents. Default credentials + no encryption = trivial exfiltration by any process on backend-net.

Incoming Communication Links: 1

Source technical asset names are clickable and link to the corresponding chapter.

MinIO File Storage (incoming)

INTENTIONAL MISCONFIGURATION: S3 API calls to MinIO over plain HTTP. Default minioadmin/minioadmin credentials are hardcoded in compose. Any container on backend-net can read all stored files.

Source: API Server
Protocol: http
Encrypted: false
Authentication: credentials
Authorization: technical-user
Read-Only: false
Usage: business
Tags: intentional-misconfiguration
VPN: false
IP-Filtered: false
Data Received: File Attachments
Data Sent: File Attachments

Nginx Reverse Proxy: 3 / 3 Risks

Description

Nginx terminates TLS and forwards all API traffic to the Express server over plain HTTP (port 3000). This is the DMZ entry point.

Identified Risks of Asset

Risk finding paragraphs are clickable and link to the corresponding chapter.

Elevated Risk Severity

Detectability: Nginx Reverse Proxy processes PII and has communication links without audit logging: Exploitation likelihood is *Likely* with *Medium* impact.

[detecting-no-audit-logging@nginx-proxy](#)

Accepted

2026-04-25

Security Team

VAULT-115

Nginx access.log captures all incoming requests (IP, URL, status code, response size). Individual API-level audit events are handled by the API server. The Nginx.API internal hop has no separate audit log; this is accepted as the TLS-termination boundary log is sufficient for DMZ-level audit requirements.

Data Disclosure: Nginx Reverse Proxy processes personal data and has an unencrypted outgoing communication link: Exploitation likelihood is *Likely* with *High* impact.

[data-disclosure-pii-unencrypted-link@nginx-proxy](#)

In Progress

2026-04-18

Security Team

VAULT-102

Duplicate of VAULT-102 (Nginx.API plaintext link). Resolved when mTLS migration completes in sprint 22. LINDDUN data-disclosure finding is tracked under the same remediation work.

Medium Risk Severity

Linkability: Nginx Reverse Proxy processes a PII data asset also flowing into another trust boundary: Exploitation likelihood is *Unlikely* with *Medium* impact.

[linking-pii-multi-boundary@nginx-proxy](#)

Accepted

2026-04-25

Security Team

VAULT-119

Nginx is the TLS-termination proxy and must bridge Internet and DMZ by design. Cross-boundary PII flow is intentional and unavoidable for this component.

Asset Information

ID:	nginx-proxy
Type:	process
Usage:	business
RAA:	1 %
Size:	service
Technology:	reverse-proxy

Tags:	nginx, tls-termination
Internet:	false
Machine:	container
Encryption:	none
Multi-Tenant:	false
Redundant:	false
Custom-Developed:	false
Client by Human:	false
Data Processed:	File Attachments, Note Content, User Credentials
Data Stored:	none
Formats Accepted:	JSON

Asset Rating

Owner:	VaultNote Team
Confidentiality:	strictly-confidential (rated 5 in scale of 5)
Integrity:	critical (rated 4 in scale of 5)
Availability:	critical (rated 4 in scale of 5)
CIA-Justification:	Nginx is the single external entry point. Its compromise exposes all downstream services. Availability is critical — outage = full service outage.

Outgoing Communication Links: 1

Target technical asset names are clickable and link to the corresponding chapter.

HTTP to API Server (outgoing)

INTENTIONAL MISCONFIGURATION: Traffic between Nginx and the API server uses plaintext HTTP. An attacker with access to the Docker network (container escape, compromised sidecar) can intercept credentials, session tokens, and note content in transit.

Target:	API Server
Protocol:	http
Encrypted:	false
Authentication:	none
Authorization:	none
Read-Only:	false
Usage:	business
Tags:	intentional-misconfiguration
VPN:	false

IP-Filtered:	false
Data Sent:	File Attachments, Note Content, User Credentials
Data Received:	File Attachments, Note Content

Incoming Communication Links: 1

Source technical asset names are clickable and link to the corresponding chapter.

HTTPS to Nginx (incoming)

User-initiated HTTPS requests carrying credentials (login) and note content (create/edit). This is the only encrypted link in the chain.

Source:	Browser SPA
Protocol:	https
Encrypted:	true
Authentication:	credentials
Authorization:	end-user-identity-propagation
Read-Only:	false
Usage:	business
Tags:	none
VPN:	false
IP-Filtered:	false
Data Received:	File Attachments, Note Content, User Credentials
Data Sent:	File Attachments, Note Content

Note RAG Pipeline: 2 / 2 Risks

Description

Retrieval-Augmented Generation pipeline that assembles context chunks from the vector store before forwarding to the LLM. Built on LangChain. INTENTIONAL: No input validation or injection guard on retrieved chunks (has-input-validation absent). A note containing prompt injection payloads could be retrieved as context and alter LLM behavior. No retrieval filtering — all results from the vector store are injected into context without relevance threshold or content sanitisation.

Identified Risks of Asset

Risk finding paragraphs are clickable and link to the corresponding chapter.

Elevated Risk Severity

Detectability: Note RAG Pipeline processes PII and has communication links without audit logging: Exploitation likelihood is *Likely* with *Medium* impact.

[detecting-no-audit-logging@rag-pipeline](#)

Unchecked

Medium Risk Severity

Linkability: Note RAG Pipeline processes a PII data asset also flowing into another trust boundary: Exploitation likelihood is *Unlikely* with *Medium* impact.

[linking-pii-multi-boundary@rag-pipeline](#)

Unchecked

Asset Information

ID:	rag-pipeline
Type:	process
Usage:	business
RAA:	11 %
Size:	component
Technology:	rag-pipeline
Tags:	ai, langchain, rag
Internet:	false
Machine:	container
Encryption:	transparent
Multi-Tenant:	false
Redundant:	false

Custom-Developed: true
Client by Human: false
Data Processed: Embedding Vectors, Note Content
Data Stored: none
Formats Accepted: JSON

Asset Rating

Owner: VaultNote Team
Confidentiality: confidential (rated 4 in scale of 5)
Integrity: critical (rated 4 in scale of 5)
Availability: operational (rated 2 in scale of 5)
CIA-Justification: RAG pipeline is the injection point between user-controlled note content and the LLM context window. Without input validation, a malicious note can manipulate the LLM into ignoring instructions, leaking system prompts, or generating harmful content on behalf of an attacker.

Outgoing Communication Links: 1

Target technical asset names are clickable and link to the corresponding chapter.

Retrieve from Vector Store (outgoing)

RAG pipeline queries the vector store for nearest-neighbor embeddings to build context chunks. Returns raw embedding text — no content filtering before injection into LLM context.

Target: VaultNote Vector Store
Protocol: https
Encrypted: true
Authentication: credentials
Authorization: technical-user
Read-Only: true
Usage: business
Tags: none
VPN: false
IP-Filtered: false
Data Sent: Embedding Vectors
Data Received: Note Content

Incoming Communication Links: 1

Source technical asset names are clickable and link to the corresponding chapter.

Call RAG Pipeline (incoming)

LLM inference endpoint triggers the RAG pipeline to build context from stored note embeddings before generating a summary response.

Source:	LLM Note Summarizer
Protocol:	https
Encrypted:	true
Authentication:	credentials
Authorization:	technical-user
Read-Only:	false
Usage:	business
Tags:	none
VPN:	false
IP-Filtered:	false
Data Received:	Note Content
Data Sent:	Note Content

PostgreSQL Database: 2 / 2 Risks

Description

Primary relational datastore. Contains users table (email + bcrypt password hash), notes table (title + content), and attachments metadata. INTENTIONAL MISCONFIGURATION: No encryption at rest (encryption: none).

Identified Risks of Asset

Risk finding paragraphs are clickable and link to the corresponding chapter.

Elevated Risk Severity

Identifiability: datastore **PostgreSQL Database** stores personal data without encryption at rest: Exploitation likelihood is *Likely* with *High* impact.

[identifying-pii-stored-unencrypted@postgresql-db](#)

Unchecked

Medium Risk Severity

Linkability: **PostgreSQL Database** processes a PII data asset also flowing into another trust boundary: Exploitation likelihood is *Unlikely* with *Medium* impact.

[linking-pii-multi-boundary@postgresql-db](#)

Accepted

2026-04-25 Security Team

VAULT-119

PostgreSQL stores credentials and note content that originate from multiple trust boundaries (Internet . DMZ . App Network . Data Tier). This is inherent to the data flow by design. Risk is accepted; mitigated by parameterized queries and authentication-only access via the API.

Asset Information

ID:	postgresql-db
Type:	datastore
Usage:	business
RAA:	54 %
Size:	service
Technology:	database
Tags:	criticality-high, information-asset-container, postgresql, relational, third-party-shared
Internet:	false
Machine:	container
Encryption:	none
Multi-Tenant:	false

Redundant: false
Custom-Developed: false
Client by Human: false
Data Processed: Note Content, User Credentials
Data Stored: Note Content, User Credentials
Formats Accepted: JSON

Asset Rating

Owner: VaultNote Team
Confidentiality: strictly-confidential (rated 5 in scale of 5)
Integrity: critical (rated 4 in scale of 5)
Availability: critical (rated 4 in scale of 5)
CIA-Justification: Holds all user PII (email, hashed passwords) and all note content. Strictly confidential — unencrypted disk access exposes all data.

Incoming Communication Links: 1

Source technical asset names are clickable and link to the corresponding chapter.

PostgreSQL Connection (incoming)

Application-level SQL queries for user auth, notes CRUD. Uses parameterized queries (pg library) to prevent SQL injection. No TLS on the connection — traffic is plaintext within backend-net.

Source: API Server
Protocol: sql-access-protocol
Encrypted: false
Authentication: credentials
Authorization: technical-user
Read-Only: false
Usage: business
Tags: none
VPN: false
IP-Filtered: false
Data Received: Note Content, User Credentials
Data Sent: Note Content, User Credentials

Redis Cache: 2 / 2 Risks

Description

In-memory store used as session store and JWT revocation list. INTENTIONAL MISCONFIGURATION: No encryption at rest. If Redis persistence is enabled and the RDB file is exfiltrated, session tokens are exposed.

Identified Risks of Asset

Risk finding paragraphs are clickable and link to the corresponding chapter.

Elevated Risk Severity

Identifiability: datastore **Redis Cache** stores personal data without encryption at rest: Exploitation likelihood is *Likely* with *High* impact.

[identifying-pii-stored-unencrypted@redis-cache](#)

Unchecked

Medium Risk Severity

Linkability: **Redis Cache** processes a PII data asset also flowing into another trust boundary: Exploitation likelihood is *Unlikely* with *Medium* impact.

[linking-pii-multi-boundary@redis-cache](#)

Accepted

2026-04-25 Security Team

VAULT-119

Session identifiers originate from users across the Internet boundary. Storage in Redis within the Data Tier is unavoidable. Mitigated by short TTL (15 minutes), revocation list, and password-protected access.

Asset Information

ID:	redis-cache
Type:	datastore
Usage:	business
RAA:	22 %
Size:	component
Technology:	database
Tags:	cache, criticality-high, information-asset-container, redis, session-store
Internet:	false
Machine:	container
Encryption:	none
Multi-Tenant:	false
Redundant:	false
Custom-Developed:	false

Client by Human: false
Data Processed: Session Tokens
Data Stored: Session Tokens
Formats Accepted: none of the special data formats accepted

Asset Rating

Owner: VaultNote Team
Confidentiality: confidential (rated 4 in scale of 5)
Integrity: critical (rated 4 in scale of 5)
Availability: important (rated 3 in scale of 5)
CIA-Justification: Session token exposure enables account takeover. Non-persistent by default but the container volume could persist RDB snapshots.

Incoming Communication Links: 1

Source technical asset names are clickable and link to the corresponding chapter.

Redis Session Store (incoming)

Session token storage and JWT revocation list. Password-protected (requirepass) but no TLS.

Source: API Server
Protocol: nosql-access-protocol
Encrypted: false
Authentication: credentials
Authorization: technical-user
Read-Only: false
Usage: business
Tags: none
VPN: false
IP-Filtered: false
Data Received: Session Tokens
Data Sent: Session Tokens

VaultNote Vector Store: 2 / 2 Risks

Description

Qdrant vector database storing embedding vectors for semantic search and RAG context retrieval. Vectors are generated by the embedding service from user note content. INTENTIONAL: No encryption at rest configured (encryption: none). The vector store is queryable by the LLM pipeline and the API server without row-level isolation — all user embeddings share the same collection. No access control list separates per-user embedding queries.

Identified Risks of Asset

Risk finding paragraphs are clickable and link to the corresponding chapter.

Elevated Risk Severity

Identifiability: datastore **VaultNote Vector Store** stores personal data without encryption at rest: Exploitation likelihood is *Likely with High* impact.

[identifying-pii-stored-unencrypted@vector-store](#)

Unchecked

Medium Risk Severity

Linkability: **VaultNote Vector Store** processes a PII data asset also flowing into another trust boundary: Exploitation likelihood is *Unlikely with Medium* impact.

[linking-pii-multi-boundary@vector-store](#)

Unchecked

Asset Information

ID:	vector-store
Type:	datastore
Usage:	business
RAA:	26 %
Size:	service
Technology:	vector-store
Tags:	ai, criticality-high, embeddings, information-asset-container, qdrant
Internet:	false
Machine:	container
Encryption:	none
Multi-Tenant:	false
Redundant:	false

Custom-Developed: false
Client by Human: false
Data Processed: Embedding Vectors, Note Content
Data Stored: Embedding Vectors
Formats Accepted: JSON

Asset Rating

Owner: VaultNote Team
Confidentiality: confidential (rated 4 in scale of 5)
Integrity: operational (rated 2 in scale of 5)
Availability: operational (rated 2 in scale of 5)
CIA-Justification: Embedding vectors encode semantic information from user notes. Even without raw text, vectors can be used to infer content via inversion attacks. Unencrypted storage and lack of per-user access control create cross-tenant leakage risk.

Incoming Communication Links: 2

Source technical asset names are clickable and link to the corresponding chapter.

Query Vector Store (incoming)

LLM retrieves relevant note chunks from the vector store to build RAG context. Similarity search over user-owned embeddings.

Source: LLM Note Summarizer
Protocol: https
Encrypted: true
Authentication: credentials
Authorization: technical-user
Read-Only: true
Usage: business
Tags: none
VPN: false
IP-Filtered: false
Data Received: Embedding Vectors
Data Sent: Note Content

Retrieve from Vector Store (incoming)

RAG pipeline queries the vector store for nearest-neighbor embeddings to build context chunks. Returns raw embedding text — no content filtering before injection into LLM context.

Source:	Note RAG Pipeline
Protocol:	https
Encrypted:	true
Authentication:	credentials
Authorization:	technical-user
Read-Only:	true
Usage:	business
Tags:	none
VPN:	false
IP-Filtered:	false
Data Received:	Embedding Vectors
Data Sent:	Note Content

AWS API Gateway: 1 / 1 Risk

Description

AWS API Gateway (HTTP API) fronting the ECS API containers in production. Routes traffic from the ALB to backend ECS tasks. INTENTIONAL: No usage plan or throttling policy configured. No WAF association on the API gateway stage. Auth relies entirely on the API server (JWT) — no gateway-level auth enforcement.

Identified Risks of Asset

Risk finding paragraphs are clickable and link to the corresponding chapter.

Medium Risk Severity

Linkability: AWS API Gateway processes a PII data asset also flowing into another trust boundary: Exploitation likelihood is *Unlikely* with *Medium* impact.

[linking-pii-multi-boundary@aws-api-gateway](#)

Unchecked

Asset Information

ID:	aws-api-gateway
Type:	process
Usage:	business
RAA:	30 %
Size:	service
Technology:	cloud-api-gateway
Tags:	api-gateway, aws, cloud-native
Internet:	true
Machine:	virtual
Encryption:	transparent
Multi-Tenant:	false
Redundant:	true
Custom-Developed:	false
Client by Human:	false
Data Processed:	File Attachments, Note Content, User Credentials
Data Stored:	none
Formats Accepted:	JSON

Asset Rating

Owner:	VaultNote Team	
Confidentiality:	strictly-confidential	(rated 5 in scale of 5)
Integrity:	critical	(rated 4 in scale of 5)
Availability:	critical	(rated 4 in scale of 5)
CIA-Justification:	API gateway is the single internet entry point in production. No rate limiting = credential stuffing and cost amplification risk. WAF absence leaves XSS/SQLi traffic unfiltered before it reaches the API.	

Outgoing Communication Links: 1

Target technical asset names are clickable and link to the corresponding chapter.

Route to ECS API (outgoing)

HTTP forward from API Gateway to the ECS-hosted API containers via ALB. Traffic is within the AWS VPC — no public routing for backend traffic.

Target:	API Server
Protocol:	https
Encrypted:	true
Authentication:	none
Authorization:	none
Read-Only:	false
Usage:	business
Tags:	none
VPN:	false
IP-Filtered:	true
Data Sent:	File Attachments, Note Content, User Credentials
Data Received:	File Attachments, Note Content

AWS ECR Container Registry: 0 / 0 Risks

Description

AWS Elastic Container Registry storing VaultNote Docker images (api, nginx). INTENTIONAL: Image vulnerability scanning is disabled (ECR Basic scanning only, not Enhanced/Inspector). Images are not signed — no Cosign/Sigstore policy enforced. Repository is private but no admission controller validates signatures at deploy time.

Identified Risks of Asset

No risksStr were identified.

Asset Information

ID:	container-registry
Type:	datastore
Usage:	devops
RAA:	9 %
Size:	service
Technology:	container-registry
Tags:	aws, ecr, supply-chain
Internet:	true
Machine:	virtual
Encryption:	transparent
Multi-Tenant:	false
Redundant:	false
Custom-Developed:	false
Client by Human:	false
Data Processed:	Build Artifacts
Data Stored:	Build Artifacts
Formats Accepted:	File

Asset Rating

Owner:	VaultNote Team	
Confidentiality:	confidential	(rated 4 in scale of 5)
Integrity:	critical	(rated 4 in scale of 5)
Availability:	operational	(rated 2 in scale of 5)

CIA-Justification: Container registry is the final source of truth for deployed images. Tampering with registry contents (image swap or layer injection) propagates malicious code to every deployment. No signing verification means a registry compromise is undetectable until damage is done.

Incoming Communication Links: 2

Source technical asset names are clickable and link to the corresponding chapter.

Push Image to Registry (incoming)

docker push to AWS ECR after build completes. Images are tagged with the git commit SHA and pushed without cryptographic signing (no Cosign step).

Source:	GitHub Actions Build Pipeline
Protocol:	https
Encrypted:	true
Authentication:	credentials
Authorization:	technical-user
Read-Only:	false
Usage:	devops
Tags:	none
VPN:	false
IP-Filtered:	false
Data Received:	Build Artifacts
Data Sent:	none

Pull Images from ECR (incoming)

ECS task pulls Docker images from ECR at task launch. Images are not verified against signatures before execution.

Source:	ECS Container Platform
Protocol:	https
Encrypted:	true
Authentication:	credentials
Authorization:	technical-user
Read-Only:	true
Usage:	devops
Tags:	none
VPN:	false
IP-Filtered:	true

Data Received: none

Data Sent: **Build Artifacts**

AWS SES: 0 / 0 Risks

Description

AWS Simple Email Service used for transactional email delivery (note-share invitations, account verification). Managed service — not custom-developed. SES sending identity (domain) is verified; DKIM and SPF records are configured. No dedicated sending quota alarm; a compromised Lambda could exhaust the daily send limit and cause notification delivery failure.

Identified Risks of Asset

No risksStr were identified.

Asset Information

ID:	aws-ses
Type:	process
Usage:	business
RAA:	0 %
Size:	service
Technology:	mail-server
Tags:	aws, cloud-native, email, ses
Internet:	true
Machine:	virtual
Encryption:	transparent
Multi-Tenant:	false
Redundant:	true
Custom-Developed:	false
Client by Human:	false
Data Processed:	Notification Payloads
Data Stored:	none
Formats Accepted:	JSON

Asset Rating

Owner:	VaultNote Team	
Confidentiality:	confidential	(rated 4 in scale of 5)
Integrity:	operational	(rated 2 in scale of 5)
Availability:	operational	(rated 2 in scale of 5)

CIA-Justification: SES handles PII (email addresses). Sending quota exhaustion prevents delivery of account verification emails, blocking user registration. Email spoofing via insufficient SPF/DKIM/DMARC alignment is possible if domain records are misconfigured.

Incoming Communication Links: 1

Source technical asset names are clickable and link to the corresponding chapter.

SES Email Send (incoming)

Lambda calls AWS SES to deliver transactional emails. IAM role grants ses:SendEmail only; no SendRawEmail permission. No SES sending rate limit configured on the IAM policy — a compromised Lambda could exhaust the SES daily quota.

Source:	Lambda Email Notifier
Protocol:	https
Encrypted:	true
Authentication:	credentials
Authorization:	technical-user
Read-Only:	false
Usage:	business
Tags:	none
VPN:	false
IP-Filtered:	false
Data Received:	Notification Payloads
Data Sent:	none

GitHub Actions Build Pipeline: 0 / 0 Risks

Description

GitHub Actions CI/CD pipeline. Builds Docker images, runs tests, pushes to ECR. INTENTIONAL: No SBOM generation (no Syft or CycloneDX step configured). No build provenance attestation (no SLSA provenance step). Hardcoded AWS_SECRET_ACCESS_KEY in workflow environment variables.

Identified Risks of Asset

No risksStr were identified.

Asset Information

ID:	build-pipeline
Type:	process
Usage:	devops
RAA:	12 %
Size:	service
Technology:	build-pipeline
Tags:	ci, github-actions, supply-chain
Internet:	true
Machine:	virtual
Encryption:	transparent
Multi-Tenant:	false
Redundant:	false
Custom-Developed:	false
Client by Human:	false
Data Processed:	Build Artifacts
Data Stored:	Build Artifacts
Formats Accepted:	File

Asset Rating

Owner:	VaultNote Team	
Confidentiality:	confidential	(rated 4 in scale of 5)
Integrity:	critical	(rated 4 in scale of 5)
Availability:	operational	(rated 2 in scale of 5)

CIA-Justification: Build pipeline is a privileged component — it has read access to secrets and write access to the container registry. Compromise = full supply chain injection. No SBOM means post-build vulnerability tracking is impossible.

Outgoing Communication Links: 1

Target technical asset names are clickable and link to the corresponding chapter.

Push Image to Registry (outgoing)

docker push to AWS ECR after build completes. Images are tagged with the git commit SHA and pushed without cryptographic signing (no Cosign step).

Target:	AWS ECR Container Registry
Protocol:	https
Encrypted:	true
Authentication:	credentials
Authorization:	technical-user
Read-Only:	false
Usage:	devops
Tags:	none
VPN:	false
IP-Filtered:	false
Data Sent:	Build Artifacts
Data Received:	none

Incoming Communication Links: 1

Source technical asset names are clickable and link to the corresponding chapter.

Push to Pipeline (incoming)

Git push triggers GitHub Actions workflow. The webhook carries the commit SHA and branch reference to the CI runner.

Source:	VaultNote Source Repository
Protocol:	https
Encrypted:	true
Authentication:	credentials
Authorization:	technical-user
Read-Only:	false
Usage:	devops
Tags:	none

VPN: false
IP-Filtered: false
Data Received: Build Artifacts
Data Sent: none

VaultNote Source Repository: 0 / 0 Risks

Description

GitHub repository hosting the VaultNote monorepo (API, frontend, Terraform). Contains application secrets in compose files (minioadmin credentials visible in git history) and Terraform state references. INTENTIONAL: No branch protection rules configured — developers push directly to main, bypassing review and signed-commit requirements.

Identified Risks of Asset

No risksStr were identified.

Asset Information

ID:	source-repo
Type:	process
Usage:	business
RAA:	12 %
Size:	service
Technology:	sourcecode-repository
Tags:	git, github, supply-chain
Internet:	true
Machine:	virtual
Encryption:	transparent
Multi-Tenant:	false
Redundant:	false
Custom-Developed:	false
Client by Human:	false
Data Processed:	Build Artifacts
Data Stored:	Build Artifacts
Formats Accepted:	File

Asset Rating

Owner:	VaultNote Team	
Confidentiality:	confidential	(rated 4 in scale of 5)
Integrity:	critical	(rated 4 in scale of 5)
Availability:	operational	(rated 2 in scale of 5)

CIA-Justification: Source code is the root of the supply chain. Integrity is critical — malicious code injected here propagates to every built artifact. May contain leaked secrets.

Outgoing Communication Links: 1

Target technical asset names are clickable and link to the corresponding chapter.

Push to Pipeline (outgoing)

Git push triggers GitHub Actions workflow. The webhook carries the commit SHA and branch reference to the CI runner.

Target:	GitHub Actions Build Pipeline
Protocol:	https
Encrypted:	true
Authentication:	credentials
Authorization:	technical-user
Read-Only:	false
Usage:	devops
Tags:	none
VPN:	false
IP-Filtered:	false
Data Sent:	Build Artifacts
Data Received:	none

Identified Data Breach Probabilities by Data Asset

In total **28 potential risks** have been identified during the threat modeling process of which **0 are rated as critical, 0 as high, 16 as elevated, 12 as medium, and 0 as low.**

These risks are distributed across **8 data assets**. The following sub-chapters of this section describe the derived data breach probabilities grouped by data asset.

Technical asset names and risk IDs are clickable and link to the corresponding chapter.

AI Model Responses: 1 / 1 Risk

LLM-generated summaries and explanations of user notes. May contain paraphrased versions of sensitive note content. Cached temporarily in the API response layer.

ID:	ai-model-responses	
Usage:	business	
Quantity:	many	
Tags:	ai	
Origin:	internal	
Owner:	VaultNote Team	
Confidentiality:	confidential	(rated 4 in scale of 5)
Integrity:	operational	(rated 2 in scale of 5)
Availability:	operational	(rated 2 in scale of 5)
CIA-Justification:	LLM responses summarise user note content. If the model was manipulated via prompt injection, responses may leak or alter content from other notes.	
Processed by:	LLM Note Summarizer	
Stored by:	none	
Sent via:	none	
Received via:	none	
Data Breach:	probable	
Data Breach Risks:	This data asset has data breach potential because of 1 remaining risk:	

Probable: detecting-no-audit-logging@llm-summarizer

Build Artifacts: 2 / 2 Risks

Docker container images, compiled source bundles, and dependency lockfiles produced by the CI/CD pipeline. Integrity is critical — these are the deployable units of the application.

ID:	build-artifacts	
Usage:	devops	
Quantity:	few	
Tags:	supply-chain	
Origin:	internal	
Owner:	VaultNote Team	
Confidentiality:	internal	(rated 2 in scale of 5)
Integrity:	critical	(rated 4 in scale of 5)
Availability:	operational	(rated 2 in scale of 5)
CIA-Justification:	Build artifacts are the final deliverable of the supply chain. Integrity is critical — tampered artifacts compromise every environment they are deployed to.	
Processed by:	AWS ECR Container Registry, ECS Container Platform, GitHub Actions Build Pipeline, VaultNote Source Repository	
Stored by:	AWS ECR Container Registry, GitHub Actions Build Pipeline, VaultNote Source Repository	
Sent via:	Push to Pipeline, Push Image to Registry	
Received via:	Pull Images from ECR	
Data Breach:	probable	
Data Breach Risks:	This data asset has data breach potential because of 2 remaining risksStr:	
	Probable: detecting-no-audit-logging@ecs-platform	
	Improbable: linking-pii-multi-boundary@ecs-platform	

Embedding Vectors: 5 / 5 Risks

High-dimensional vector representations of user note content generated by the embedding service. Semantically encodes note information even without storing the original text.

ID:	embedding-vectors	
Usage:	business	
Quantity:	many	
Tags:	ai, embeddings	
Origin:	internal	
Owner:	VaultNote Team	
Confidentiality:	confidential	(rated 4 in scale of 5)
Integrity:	operational	(rated 2 in scale of 5)
Availability:	operational	(rated 2 in scale of 5)
CIA-Justification:	Embedding vectors can be used in model inversion attacks to reconstruct approximate original text. Cross-user embedding access leaks semantic note content.	
Processed by:	LLM Note Summarizer, Note RAG Pipeline, VaultNote Vector Store	
Stored by:	VaultNote Vector Store	
Sent via:	Retrieve from Vector Store, Query Vector Store	
Received via:	none	
Data Breach:	probable	
Data Breach Risks:	This data asset has data breach potential because of 5 remaining risksStr:	
	Probable: detecting-no-audit-logging@llm-summarizer	
	Probable: detecting-no-audit-logging@rag-pipeline	
	Probable: identifying-pii-stored-unencrypted@vector-store	
	Improbable: linking-pii-multi-boundary@rag-pipeline	
	Improbable: linking-pii-multi-boundary@vector-store	

File Attachments: 14 / 14 Risks

Binary file uploads attached to notes. May include documents, images, or other sensitive files depending on note usage.

ID:	file-attachments	
Usage:	business	
Quantity:	many	
Tags:	none	
Origin:	customer	
Owner:	VaultNote Team	
Confidentiality:	confidential	(rated 4 in scale of 5)
Integrity:	critical	(rated 4 in scale of 5)
Availability:	operational	(rated 2 in scale of 5)
CIA-Justification:	Files may contain sensitive documents. Integrity is critical because tampered attachments corrupt evidence or business records. MinIO's default credentials (minioadmin) make all stored files accessible to anyone on the network — intentional misconfiguration for demo purposes.	
Processed by:	API Server, AWS API Gateway, AWS S3 Notes Bucket, Browser SPA, MinIO Object Storage, Nginx Reverse Proxy	
Stored by:	AWS S3 Notes Bucket, MinIO Object Storage	
Sent via:	Route to ECS API, MinIO File Storage, HTTPS to Nginx, HTTP to API Server	
Received via:	Route to ECS API, MinIO File Storage, HTTPS to Nginx, HTTP to API Server	
Data Breach:	probable	
Data Breach Risks:	This data asset has data breach potential because of 14 remaining risksStr:	
	Probable: identifying-pii-stored-unencrypted@s3-notes-bucket	
	Probable: identifying-pii-stored-unencrypted@minio-storage	
	Probable: detecting-no-audit-logging@nginx-proxy	
	Probable: data-disclosure-pii-unencrypted-link@api-server	
	Probable: data-disclosure-pii-unencrypted-link@nginx-proxy	
	Probable: detecting-no-audit-logging@api-server	
	Possible: pii-client-side-storage@browser-spa	
	Improbable: linking-pii-multi-boundary@aws-api-gateway	
	Improbable: linking-pii-multi-boundary@s3-notes-bucket	
	Improbable: linking-pii-multi-boundary@api-server	
	Improbable: linking-pii-multi-boundary@browser-spa	
	Improbable: linking-pii-multi-boundary@minio-storage	
	Improbable: linking-pii-multi-boundary@nginx-proxy	
	Improbable: unawareness-no-consent-technology@browser-spa	

Note Content: 21 / 21 Risks

User-authored note text — may include personal, financial, or health information depending on how users employ the application.

ID:	note-content
Usage:	business
Quantity:	many
Tags:	none
Origin:	customer
Owner:	VaultNote Team
Confidentiality:	confidential (rated 4 in scale of 5)
Integrity:	critical (rated 4 in scale of 5)
Availability:	operational (rated 2 in scale of 5)
CIA-Justification:	Notes may contain highly sensitive personal information. Integrity is critical because corrupted notes lose user trust.
Processed by:	API Server, AWS API Gateway, AWS RDS PostgreSQL, Browser SPA, ECS Container Platform, LLM Note Summarizer, Nginx Reverse Proxy, Note RAG Pipeline, PostgreSQL Database, VaultNote Vector Store
Stored by:	AWS RDS PostgreSQL, PostgreSQL Database
Sent via:	Route to ECS API, PostgreSQL Connection, HTTPS to Nginx, HTTP to API Server, Call RAG Pipeline
Received via:	Route to ECS API, Retrieve from Vector Store, Query Vector Store, PostgreSQL Connection, HTTPS to Nginx, HTTP to API Server, Call RAG Pipeline
Data Breach:	probable
Data Breach Risks:	This data asset has data breach potential because of 21 remaining risksStr: - Probable: detecting-no-audit-logging@ecs-platform - Probable: detecting-no-audit-logging@llm-summarizer - Probable: detecting-no-audit-logging@rag-pipeline - Probable: identifying-pii-stored-unencrypted@rds-postgresql - Probable: identifying-pii-stored-unencrypted@postgresql-db - Probable: identifying-pii-stored-unencrypted@vector-store - Probable: detecting-no-audit-logging@nginx-proxy - Probable: data-disclosure-pii-unencrypted-link@api-server - Probable: data-disclosure-pii-unencrypted-link@nginx-proxy - Probable: detecting-no-audit-logging@api-server - Possible: pii-client-side-storage@browser-spa - Improbable: linking-pii-multi-boundary@aws-api-gateway - Improbable: linking-pii-multi-boundary@rds-postgresql - Improbable: linking-pii-multi-boundary@ecs-platform - Improbable: linking-pii-multi-boundary@rag-pipeline - Improbable: linking-pii-multi-boundary@vector-store

Improbable: linking-pii-multi-boundary@api-server

Improbable: linking-pii-multi-boundary@browser-spa

Improbable: linking-pii-multi-boundary@nginx-proxy

Improbable: linking-pii-multi-boundary@postgresql-db

Improbable: unawareness-no-consent-technology@browser-spa

Notification Payloads: 1 / 1 Risk

Email notification payloads containing user email addresses and note titles sent via Lambda to AWS SES. Contains PII (email address) and indirect note content references.

ID:	notification-payloads	
Usage:	business	
Quantity:	many	
Tags:	pii	
Origin:	internal	
Owner:	VaultNote Team	
Confidentiality:	confidential	(rated 4 in scale of 5)
Integrity:	operational	(rated 2 in scale of 5)
Availability:	operational	(rated 2 in scale of 5)
CIA-Justification:	Email addresses are PII. Note titles may be sensitive. Loss via DLQ absence creates unauditable data processing gaps.	
Processed by:	AWS SES, Lambda Email Notifier	
Stored by:	none	
Sent via:	SES Email Send	
Received via:	none	
Data Breach:	probable	
Data Breach Risks:	This data asset has data breach potential because of 1 remaining risk:	
	Probable: detecting-no-audit-logging@lambda-notifier	

Session Tokens: 5 / 5 Risks

JWT tokens and Redis session keys used to maintain authenticated state. Theft allows session hijacking without knowing the user's password.

ID:	session-tokens	
Usage:	business	
Quantity:	many	
Tags:	none	
Origin:	internal	
Owner:	VaultNote Team	
Confidentiality:	confidential	(rated 4 in scale of 5)
Integrity:	critical	(rated 4 in scale of 5)
Availability:	operational	(rated 2 in scale of 5)
CIA-Justification:	Session token compromise enables full session hijacking. Short TTLs and revocation lists mitigate but do not eliminate the risk.	
Processed by:	API Server, Redis Cache	
Stored by:	Redis Cache	
Sent via:	Redis Session Store	
Received via:	Redis Session Store	
Data Breach:	probable	
Data Breach Risks:	This data asset has data breach potential because of 5 remaining risksStr:	

Probable: identifying-pii-stored-unencrypted@redis-cache

Probable: data-disclosure-pii-unencrypted-link@api-server

Probable: detecting-no-audit-logging@api-server

Improbable: linking-pii-multi-boundary@api-server

Improbable: linking-pii-multi-boundary@redis-cache

User Credentials: 16 / 16 Risks

Email addresses and bcrypt-hashed passwords used for authentication. Compromise allows full account takeover.

ID:	user-credentials
Usage:	business
Quantity:	many
Tags:	none
Origin:	customer
Owner:	VaultNote Team
Confidentiality:	strictly-confidential (rated 5 in scale of 5)
Integrity:	critical (rated 4 in scale of 5)
Availability:	operational (rated 2 in scale of 5)
CIA-Justification:	Credentials grant full account access. A breach leaks all users' identities and enables account takeover across every note and attachment.
Processed by:	API Server, AWS API Gateway, AWS RDS PostgreSQL, Browser SPA, ECS Container Platform, Nginx Reverse Proxy, PostgreSQL Database
Stored by:	AWS RDS PostgreSQL, PostgreSQL Database
Sent via:	Route to ECS API, PostgreSQL Connection, HTTPS to Nginx, HTTP to API Server
Received via:	PostgreSQL Connection
Data Breach:	probable
Data Breach Risks:	This data asset has data breach potential because of 16 remaining risksStr:

Probable: detecting-no-audit-logging@ecs-platform

Probable: identifying-pii-stored-unencrypted@rds-postgresql

Probable: identifying-pii-stored-unencrypted@postgresql-db

Probable: detecting-no-audit-logging@nginx-proxy

Probable: data-disclosure-pii-unencrypted-link@api-server

Probable: data-disclosure-pii-unencrypted-link@nginx-proxy

Probable: detecting-no-audit-logging@api-server

Possible: pii-client-side-storage@browser-spa

Improbable: linking-pii-multi-boundary@aws-api-gateway

Improbable: linking-pii-multi-boundary@rds-postgresql

Improbable: linking-pii-multi-boundary@ecs-platform

Improbable: linking-pii-multi-boundary@api-server

Improbable: linking-pii-multi-boundary@browser-spa

Improbable: linking-pii-multi-boundary@nginx-proxy

Improbable: linking-pii-multi-boundary@postgresql-db

Improbable: unawareness-no-consent-technology@browser-spa

Trust Boundaries

In total **7 trust boundaries** have been modeled during the threat modeling process.

AI Services

Internal network housing the AI/ML components (vector store, RAG pipeline). The LLM inference endpoint is internet-accessible — it is in the Internet trust boundary. AI components have outbound access to the vector store and receive note content from the API server.

ID: ai-services
Type: network-on-prem
Tags: ai
Assets inside: Note RAG Pipeline, VaultNote Vector Store
Boundaries nested: none

AWS Cloud

AWS production environment (VPC). Contains managed services (RDS, S3, ECS, Lambda, API Gateway). Separated from the dev Docker environment by an air gap — production traffic flows through the API Gateway, never through the Docker compose setup.

ID: aws-cloud
Type: network-cloud-provider
Tags: aws, cloud-native
Assets inside: AWS API Gateway, AWS SES, ECS Container Platform, Lambda Email Notifier, AWS RDS PostgreSQL, AWS S3 Notes Bucket
Boundaries nested: none

Application Network

Internal Docker bridge network (frontend-net). Contains the API server which is reachable from Nginx but also bridges into backend-net.

ID: application-network
Type: network-on-prem
Tags: none
Assets inside: API Server
Boundaries nested: none

DMZ

Demilitarized zone containing the Nginx reverse proxy. Accessible from the internet but isolated from the application and data tiers.

ID: dmz
Type: network-on-prem
Tags: none
Assets inside: Nginx Reverse Proxy
Boundaries nested: none

Data Tier

Internal Docker bridge network (backend-net, marked internal: true). No external routing. Contains all datastores. The API server has a NIC on this network and acts as the sole legitimate access path.

ID: data-tier
Type: network-on-prem
Tags: none
Assets inside: MinIO Object Storage, PostgreSQL Database, Redis Cache
Boundaries nested: none

External CI/CD

GitHub-hosted CI/CD infrastructure and AWS ECR registry. External to the application runtime but directly connected to it via image pull at deploy time. Trust boundary represents the entire supply chain attack surface.

ID: external-cicd
Type: network-cloud-provider
Tags: supply-chain
Assets inside: GitHub Actions Build Pipeline, AWS ECR Container Registry, VaultNote Source Repository
Boundaries nested: none

Internet

Untrusted external network. All traffic from this zone must cross the Nginx TLS termination point before entering the DMZ.

ID: internet

Type: [network-on-prem](#)
Tags: none
Assets inside: **Browser SPA**
Boundaries nested: none

Shared Runtimes

In total **1 shared runtime** has been modeled during the threat modeling process.

Docker Host

All containers run on the same Docker host. A container escape or privileged container could access the host filesystem including Docker socket and database volumes.

ID:	docker-host
Tags:	container-runtime, docker
Assets running:	Nginx Reverse Proxy, API Server, PostgreSQL Database, Redis Cache, MinIO Object Storage

Risk Rules Checked by Threagile

Threagile Version: 1.0.0

Threagile Build Timestamp:

Threagile Execution Timestamp: 20260604113020

Model Filename: threagile.yaml

Model Hash (SHA256): 15d123b13753140c59d8c6698da83a704aebcacf211732f0ee2ef3b5931b60ef

Threagile (see <https://threagile.io> for more details) is an open-source toolkit for agile threat modeling, created by Christian Schneider (<https://christian-schneider.net>): It allows to model an architecture with its assets in an agile fashion as a YAML file directly inside the IDE. Upon execution of the Threagile toolkit all standard risk rules (as well as individual custom rules if present) are checked against the architecture model. At the time the Threagile toolkit was executed on the model input file the following risk rules were checked:

Disclaimer

VaultNote Security Team conducted this threat analysis using the open-source Threagile toolkit on the applications and systems that were modeled as of this report's date. Information security threats are continually changing, with new vulnerabilities discovered on a daily basis, and no application can ever be 100% secure no matter how much threat modeling is conducted. It is recommended to execute threat modeling and also penetration testing on a regular basis (for example yearly) to ensure a high ongoing level of security and constantly check for new attack vectors.

This report cannot and does not protect against personal or business loss as the result of use of the applications or systems described. VaultNote Security Team and the Threagile toolkit offers no warranties, representations or legal certifications concerning the applications or systems it tests. All software includes defects: nothing in this document is intended to represent or warrant that threat modeling was complete and without error, nor does this document represent or warrant that the architecture analyzed is suitable to task, free of other defects than reported, fully compliant with any industry standards, or fully compatible with any operating system, hardware, or other application. Threat modeling tries to analyze the modeled architecture without having access to a real working system and thus cannot and does not test the implementation for defects and vulnerabilities. These kinds of checks would only be possible with a separate code review and penetration test against a working system and not via a threat model.

By using the resulting information you agree that VaultNote Security Team and the Threagile toolkit shall be held harmless in any event.

This report is confidential and intended for internal, confidential use by the client. The recipient is obligated to ensure the highly confidential contents are kept secret. The recipient assumes responsibility for further distribution of this document.

In this particular project, a time box approach was used to define the analysis effort. This means that the author allotted a prearranged amount of time to identify and document threats. Because of this, there is no guarantee that all possible threats and risks are discovered. Furthermore, the analysis applies to a snapshot of the current state of the modeled architecture (based on the architecture information provided by the customer) at the examination time.

Report Distribution

Distribution of this report (in full or in part like diagrams or risk findings) requires that this disclaimer as well as the chapter about the Threagile toolkit and method used is kept intact as part of the distributed report or referenced from the distributed parts.